

# Reinforcement Learning from Verifiable Rewards

Kian Kyars

# Table of contents

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Start Here</b>                                   | <b>3</b>  |
| Abstract . . . . .                                  | 3         |
| New to RLVR . . . . .                               | 4         |
| Building Systems . . . . .                          | 4         |
| Frontier Research . . . . .                         | 4         |
| LLM Use . . . . .                                   | 4         |
| Target Audience . . . . .                           | 4         |
| How to Use This Book . . . . .                      | 4         |
| Changelog . . . . .                                 | 5         |
| Acknowledgments . . . . .                           | 5         |
| Github Contributors . . . . .                       | 5         |
| Citation . . . . .                                  | 5         |
| <br>                                                |           |
| <b>I Foundations</b>                                | <b>6</b>  |
| <br>                                                |           |
| <b>1 Introduction</b>                               | <b>7</b>  |
| 1.1 Chapter Map . . . . .                           | 8         |
| 1.2 What RLVR Is . . . . .                          | 8         |
| 1.3 Origins of RLVR . . . . .                       | 8         |
| 1.4 Verifiable Tasks . . . . .                      | 9         |
| 1.5 RLVR and Reasoning . . . . .                    | 11        |
| 1.6 Verifiable versus Complete . . . . .            | 11        |
| 1.7 What This Book Covers . . . . .                 | 11        |
| <br>                                                |           |
| <b>II Verifier Design</b>                           | <b>12</b> |
| <br>                                                |           |
| <b>2 Outcome Rewards</b>                            | <b>13</b> |
| 2.1 Chapter Map . . . . .                           | 13        |
| 2.2 A Rollout . . . . .                             | 14        |
| 2.3 How outcome verifiers are implemented . . . . . | 14        |
| 2.4 A minimal outcome verifier . . . . .            | 15        |
| 2.5 Outcome check, full-trajectory update . . . . . | 17        |
| 2.6 Domain specific considerations . . . . .        | 18        |
| 2.7 Brittleness . . . . .                           | 19        |
| 2.8 Open questions . . . . .                        | 19        |
| <br>                                                |           |
| <b>3 Process Rewards</b>                            | <b>20</b> |
| 3.1 Chapter Map . . . . .                           | 20        |
| 3.2 The credit assignment problem . . . . .         | 21        |
| 3.3 What a process reward model computes . . . . .  | 22        |
| 3.4 How step labels are obtained . . . . .          | 22        |
| 3.5 ORM vs PRM . . . . .                            | 24        |
| 3.6 Limitations . . . . .                           | 24        |
| 3.7 Open questions . . . . .                        | 25        |

|            |                                                    |           |
|------------|----------------------------------------------------|-----------|
| <b>4</b>   | <b>Learned, Programmatic, and Hybrid Verifiers</b> | <b>26</b> |
| 4.1        | Chapter Map                                        | 26        |
| 4.2        | Programmatic versus Learned Verifiers              | 26        |
| 4.3        | Programmatic verifiers                             | 26        |
| 4.4        | Learned verifiers                                  | 27        |
| 4.5        | Hybrid stacks                                      | 28        |
| 4.6        | Formalization                                      | 29        |
| 4.7        | Limitations                                        | 30        |
| 4.8        | Open questions                                     | 30        |
| 4.9        | What comes next                                    | 30        |
| <b>III</b> | <b>From Verifiers to Capability</b>                | <b>31</b> |
| <b>5</b>   | <b>Turning Checks into Training Signal</b>         | <b>33</b> |
| 5.1        | Chapter Map                                        | 34        |
| 5.2        | A compact outcome-RLVR training script             | 34        |
| 5.3        | Reward engineering                                 | 38        |
| 5.4        | Case study                                         | 42        |
| 5.5        | Open questions                                     | 43        |
| 5.6        | What comes next                                    | 43        |
| <b>6</b>   | <b>Search and Test Time Verification</b>           | <b>44</b> |
| 6.1        | Chapter Map                                        | 44        |
| 6.2        | Test Time                                          | 45        |
| 6.3        | Selection                                          | 45        |
| 6.4        | Search: verifier as controller                     | 47        |
| 6.5        | Amortization                                       | 49        |
| 6.6        | Reporting results                                  | 49        |
| 6.7        | Open questions                                     | 49        |
| 6.8        | What comes next                                    | 49        |
| <b>IV</b>  | <b>Optimization Pressure</b>                       | <b>50</b> |
| <b>7</b>   | <b>Reward Hacking</b>                              | <b>51</b> |
| 7.1        | Chapter Map                                        | 51        |
| 7.2        | Goodhart's Law                                     | 51        |
| 7.3        | A taxonomy of verifier exploits                    | 52        |
| 7.4        | Empirical exploits                                 | 53        |
| 7.5        | The over-optimization curve                        | 54        |
| 7.6        | Tail precision                                     | 55        |
| 7.7        | Test time exploits                                 | 56        |
| 7.8        | Hardening techniques                               | 56        |
| 7.9        | Open questions                                     | 56        |

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| <b>V</b>  | <b>Domains and Frontiers</b>                          | <b>57</b> |
| <b>8</b>  | <b>A Frontier Recipe</b>                              | <b>58</b> |
| 8.1       | Chapter Map . . . . .                                 | 58        |
| 8.2       | Setup . . . . .                                       | 58        |
| 8.3       | Post-training . . . . .                               | 58        |
| 8.4       | GRPO . . . . .                                        | 59        |
| 8.5       | Rewards . . . . .                                     | 59        |
| 8.6       | Filtering and data mixing . . . . .                   | 60        |
| 8.7       | The rollout system . . . . .                          | 60        |
| 8.8       | PipelineRL . . . . .                                  | 60        |
| 8.9       | Takeaways . . . . .                                   | 61        |
| <b>9</b>  | <b>Long-context, multimodal, and agentic RLVR</b>     | <b>62</b> |
| 9.1       | Chapter Map . . . . .                                 | 62        |
| 9.2       | From reward functions to harnesses . . . . .          | 62        |
| 9.3       | DeepSWE . . . . .                                     | 63        |
| <b>10</b> | <b>Open Problems</b>                                  | <b>65</b> |
| 10.1      | Chapter Map . . . . .                                 | 65        |
| 10.2      | Verifier fidelity beyond math and code . . . . .      | 66        |
| 10.3      | Adaptive RLVR . . . . .                               | 66        |
|           | <b>Appendices</b>                                     | <b>67</b> |
| <b>A</b>  | <b>Minimal RL and Post-Training Background</b>        | <b>67</b> |
| A.1       | Objective, States, and Rewards . . . . .              | 67        |
| A.2       | What the Optimizer Does . . . . .                     | 68        |
| A.3       | Token-Level Update Intuition . . . . .                | 68        |
| A.4       | KL Regularization . . . . .                           | 69        |
| A.5       | Rejection, Search, and Policy Optimization . . . . .  | 69        |
| <b>B</b>  | <b>Practical verifier design checklist</b>            | <b>70</b> |
| <b>C</b>  | <b>Core Terminology</b>                               | <b>71</b> |
| C.1       | Reward and verifier terms . . . . .                   | 71        |
| C.2       | Evaluation, reporting, and robustness terms . . . . . | 71        |
| C.3       | Agentic terms . . . . .                               | 72        |

## Start Here

[Open PDF](#)

[GitHub](#)



Figure 1: M. C. Escher, *Corsica Corte* (1929).

## Abstract

Reinforcement learning from verifiable rewards (RLVR) studies how models can improve by learning from reward signals derived from checkable task outcomes, executable feedback, formal validation, or other reliable forms of verification. This book's purpose is to explain what kinds of rewards can be

made verifiable, what those rewards actually train, where the paradigm has been most successful, and where it breaks.

## **New to RLVR**

Read [Chapter 1](#), [Chapter 2](#), and [Chapter 7](#).

## **Building Systems**

Read [Chapter 4](#), [Chapter 5](#), and [Chapter 9](#).

## **Frontier Research**

Read [Chapter 8](#), [Chapter 9](#), and [Chapter 10](#).

## **LLM Use**

Fortunately, we live in a world where AI slop writing is still *very* intelligible from genuine human text. It is knowing this fact, and also knowing that a textbook is still very much a human-lead endeavor, that I write (I can guarantee you there are no EM dashes in the entire book) most sections on my own, or rather use Wispr Flow to dictate them and then edit them. The main contributions of Codex to this project were:

- helping me plan out the structure
- giving me the initial boilerplate/skeleton scaffold of the textbook itself
- creating the diagrams and equations, since this is much more efficient, in particular given my lack of LaTeX scripting skills, and is inherently much lower-entropy than writing english, not requiring the same human creativity

## **Target Audience**

I wrote this book with the intent to cater to the largest audience possible. With that in mind, I increase difficulty as a function of the chapters such that if you are new to RLVR, you are best served in the beginning. If you are already experienced, you will gain the most from the later chapters.

## **How to Use This Book**

Each chapter has a TL;DR at the beginning. Although the chapters do minimally build off of each other, they can still read alone. Feel free to use the search function on the web version or Command F on the PDF to find what you wish directly. The citations are plentiful to facilitate further research if there's a specific theme which captivates you.

## Changelog

- 2026-04-16: Officially announced v0 of the book!
- 2026-04-19: Used my [ML review textbook skill](#) to refine each chapter and fix erratum.

## Acknowledgments

I shamelessly take inspiration from Nathan Lambert's [RLHF book](#), and I am well aware that his textbook treats the subject of RLVR in detail; notwithstanding, as he notes himself, this particular sub-field of ML is evolving so fast that much of the RLHF book's RLVR content will become outdated, and this book is intended to maintain pace with progress.

I also acknowledge the wonderful developers of Exclaidraw, which I used for this book's figures. Thanks to M.C. Escher for being the artistic soul of the book (FYI all art contained in this textbook is published at the latest in 1930, which means it's in the public domain in the United States). Thanks to Simon Boehm for creating amazing educational content and establishing the target I strive to reach (same for Colah from distillpub)! Lastly, thanks to the quarto devs for making the software this book is based on!

## Github Contributors

- [Yaroslav Petrashko](#)

## Citation

You can cite this book directly with this BibTeX.

```
@online{kyars2026rlvrbook,  
  title   = {Reinforcement Learning from Verifiable Rewards},  
  author  = {Kyars, Kian},  
  year    = {2026},  
  url     = {https://rlvrbook.com},  
}
```

## Part I

# Foundations



# 1 Introduction

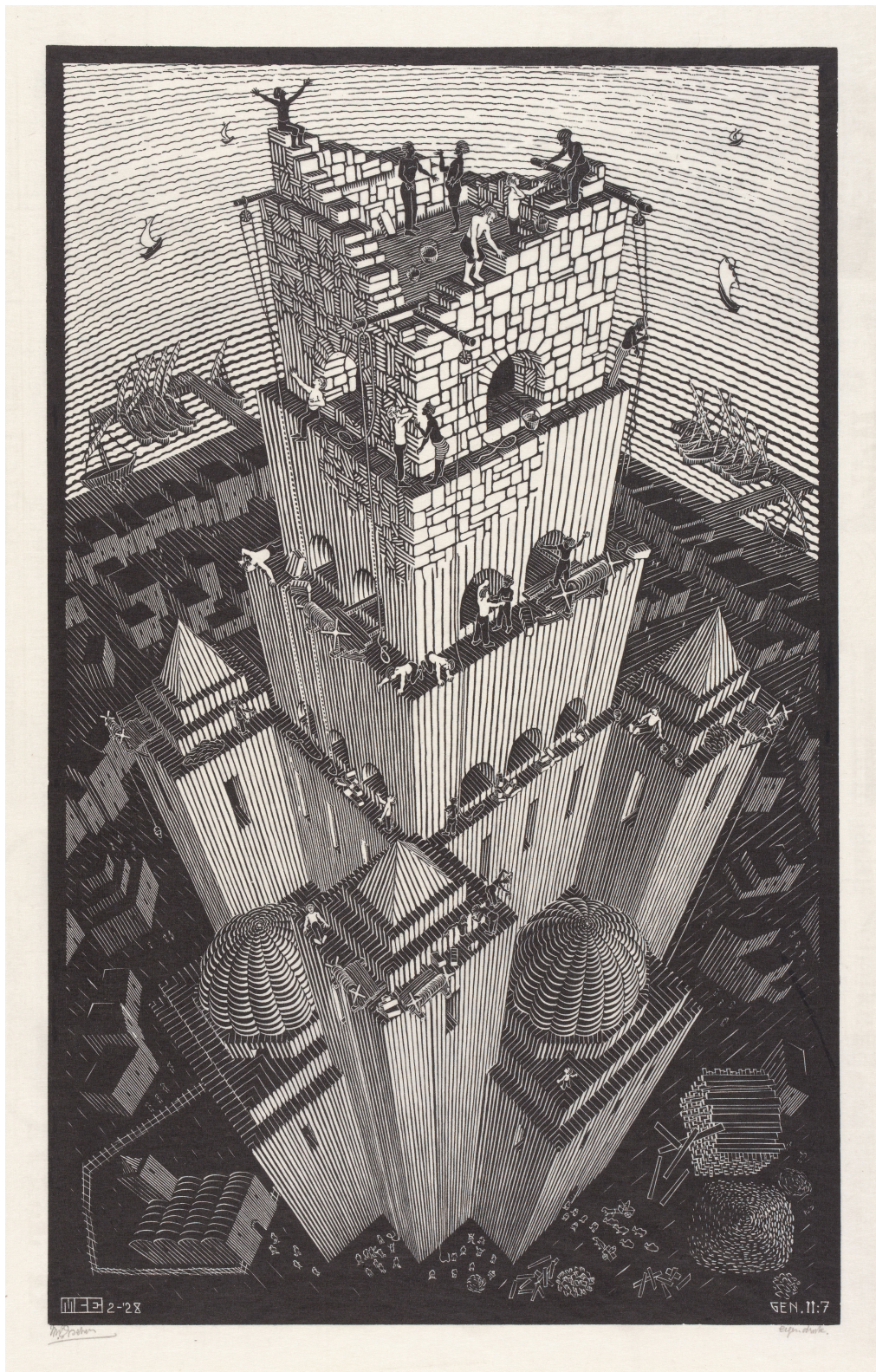


Figure 2: M. C. Escher, *Tower of Babel* (1928).

## 1.1 Chapter Map

- Define RLVR as learning from verifiable reward signals and explain verifiable tasks.
- Analyze why RLVR became central to reasoning models and preview the structure of the book.

## 1.2 What RLVR Is

RLVR is reinforcement learning on tasks where some meaningful part of correctness can be checked directly. The check you implement can be exact, as in symbolic math or formal proof, or the result of an executable, as in unit tests. It can be partial, i.e. grounded question answering or tool-using agents where only some parts of the trajectory can be reliably scored. The unifying idea is the availability of a success notion. Once a task exposes useful correctness signals, reinforcement learning can optimize against them, search can exploit them at test time, and systems can improve far beyond what static supervised fine-tuning alone produces.

We use *verifier* as the default term for the mechanism that checks output and produces a signal, related terms include *checker*, *scorer*, and sometimes *judge*.

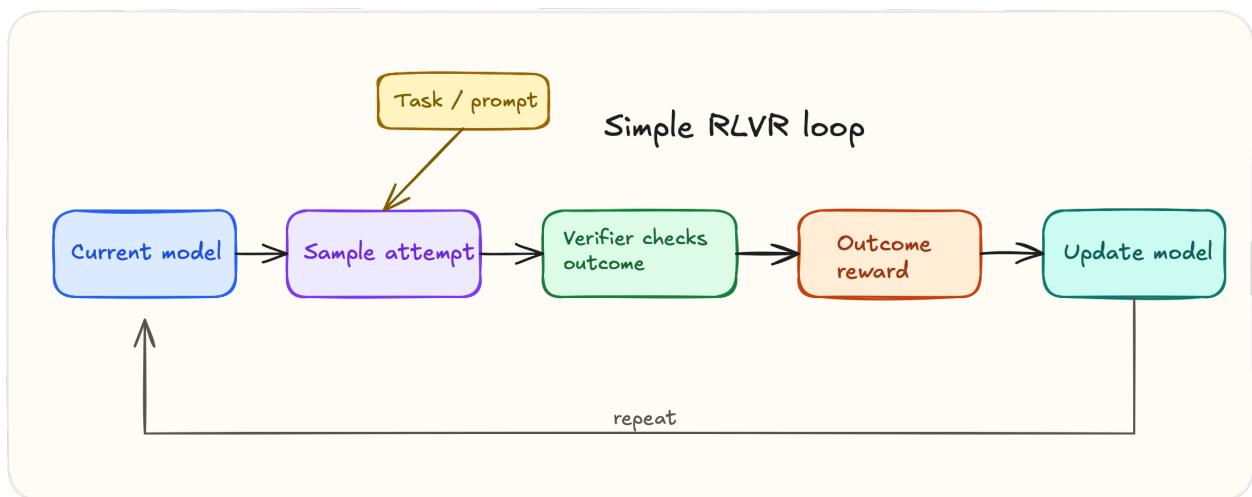


Figure 3: RLVR is defined by learning from verifiable reward signals; the optimizer can vary.

## 1.3 Origins of RLVR

In some sense RLVR is akin to the oldest paradigm in reinforcement learning, since it learns from direct reward rather than preference comparison, just like the classic RL environments, e.g. cartpole; what is new is the application to LLMs through verifiers that can check answers, code, proofs, and traces.

I personally reflect back on the advent of reasoning models and reinforcement learning through a strange amnesia of an idea so simple with hindsight, but which took two years after ChatGPT to discover. This assessment, however, is unfair in the sense that the idea to make models think step by



step long predates the 2024 reasoning-model wave.<sup>1</sup> The broader prompting paradigm emerged across late 2021 and early 2022: scratchpads for intermediate computation appeared first, chain-of-thought prompting then formalized the use of intermediate reasoning traces, and the exact prompt “Let’s think step by step” was popularized a few months later.

Before the reasoning-model wave of 2024, code generation had already explored reinforcement learning against executable verifiers: CodeRL (July 5, 2022), PPOCoder (January 31, 2023), and RLTF (July 10, 2023) all trained language models using unit tests or execution feedback as objective reward signals.<sup>2</sup>

DeepSeekMath, published on February 5, 2024, was the first major open paper to apply verifier-driven RL to mathematical reasoning at LLM scale via the introduction of GRPO.

Things heated up in September 2024, when OpenAI published “Learning to Reason with LLMs” (o1), indicating that they had used a train-time and test time compute strategy to enhance model reasoning through reinforcement learning in math, and coding tasks.<sup>3</sup> The name “Reinforcement Learning with Verifiable Rewards” (RLVR) was coined in the Tulu 3 paper from November 22, 2024.<sup>4</sup> Finally, there was DeepSeek-R1 at the start of 2025, which demonstrated the full verifier-driven RL formula for bootstrapping reasoning models.<sup>5</sup> To quote someone describing the atmosphere at Meta after R1 launched, “Engineers are moving frantically to dissect DeepSeek and copy anything and everything we can from it,” and according to Fortune, there were war rooms assembled at Meta to understand how a Chinese lab with substantially less resources was beating them.<sup>6</sup>

The trend we can extract from this short history is that model improvement increasingly depended on checkable interfaces.

## 1.4 Verifiable Tasks

Tasks admit verifiable rewards when there is an interface to separate better behavior from worse behavior at acceptable cost. Math problems allow answer checking up to normalization. Code can be run against visible and hidden tests. Formal proof systems can accept or reject proof states under explicit rules. These domains became central not because they expose unusually clean signals.

Other tasks are weaker but still useful. Long-context question answering may permit citation checks, evidence matching, or entailment-style grading. Tool-using agents have environment transitions, task

---

<sup>1</sup>A useful compressed lineage runs from scratchpads in late 2021, to chain-of-thought prompting in January 2022, to the exact zero-shot prompt “Let’s think step by step” in May 2022 (Nye et al. 2021; Wei et al. 2022; Kojima et al. 2022).

<sup>2</sup>CodeRL was submitted on July 5, 2022 and used unit tests and a critic model to guide program synthesis (Le et al. 2022). PPOCoder was submitted on January 31, 2023 and used execution-based feedback with PPO (Shojaee et al. 2023). RLTF was submitted on July 10, 2023 and used online unit-test feedback of multiple granularities for code LLMs (H. Liu et al. 2023).

<sup>3</sup>OpenAI’s writeup states that o1 performance improved with both more reinforcement learning, which they describe as train-time compute, and more time spent thinking at test time (OpenAI 2024).

<sup>4</sup>DeepSeekMath introduced GRPO and used RL to improve mathematical reasoning in an open model (Shao et al. 2024). Tulu 3 later introduced the name “Reinforcement Learning with Verifiable Rewards (RLVR)” for this broader training pattern (Lambert, Morrison, et al. 2024).

<sup>5</sup>DeepSeek-R1 argues that reasoning abilities can be incentivized through pure reinforcement learning on verifiable tasks such as mathematics, coding competitions, and STEM fields (DeepSeek-AI et al. 2025).

<sup>6</sup>The quoted line was reported as an anonymous Teamblind post summarized by TMTPOST, while the claim that Meta created four “war rooms” was reported by Fortune, citing The Information (TMTPOST Global 2025; Quiroz-Gutierrez 2025).

completion criteria, or execution traces. These signals are often noisier, more expensive, and easier to exploit, but they can still support learning if the reward channel is informative enough.

The takeaway is that there isn't a uniform notion of determining correctness across all conceivable tasks. It is strongest where correctness is legible and weakest where the reward channel is sparse, ambiguous, or only loosely coupled to the capabilities we want.

A useful way to see the space is as a domain map. One axis is verification strength: how cleanly the checker separates better behavior from worse behavior. The other is verification granularity: whether the checked object is a coarse final artifact, a partially grounded intermediate object, or a fine-grained trajectory. (Shao et al. 2024; H. Liu et al. 2023; Xin, Ren, et al. 2024; Zhang et al. 2024; Lu et al. 2023; Zhou et al. 2023; Xie et al. 2024)

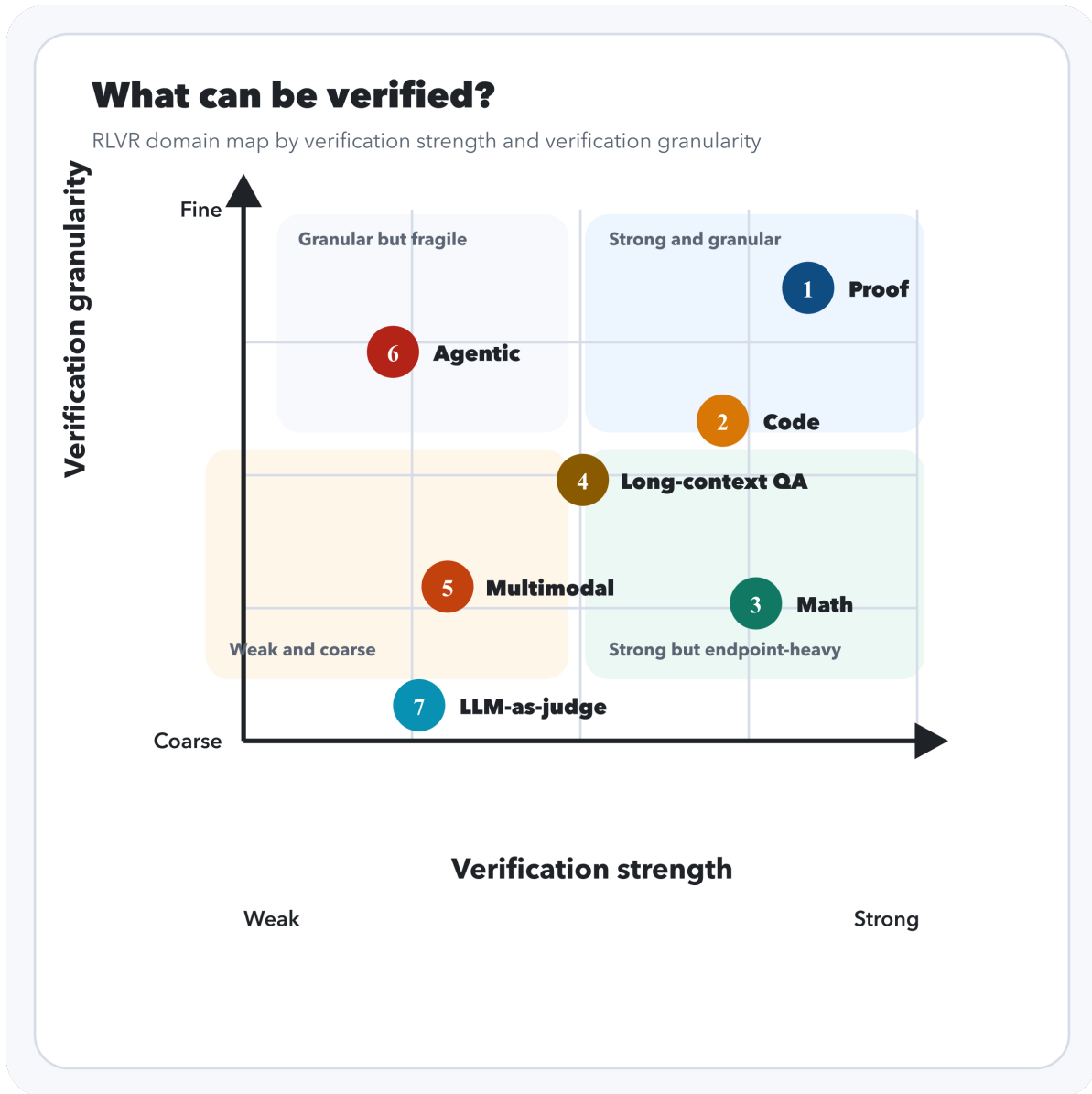


Figure 4: A domain map of RLVR by verification strength versus verification granularity.

## 1.5 RLVR and Reasoning

RLVR and reasoning go hand in hand, but they are different. The former is a training paradigm, and the latter is a downstream capability/artefact, e.g. multi-step breakdown, search, planning, tool use, etc. The marriage between the two occurs because the most successful reasoning domains are exactly the ones with strong verifiers: math, code, proofs, some grounded QA. The result is that some of the most important progress in reasoning models has come from learning against verifiable rewards. It's therefore understandable that RLVR and reasoning are conflated, since verifier-friendly domains are the best places to scale reasoning performance.

## 1.6 Verifiable versus Complete

Even verifiers are susceptible to becoming proxies, from our three core domain examples:

1. A code evaluator may miss behaviors outside the test suite.
2. A math reward may depend on brittle extraction.
3. A proof system may validate a derivation without telling us whether the model's decomposition was insightful or robust.

These examples raise important questions to consider in applying RLVR:

- what is being checked,
- what is being missed,
- how expensive is the check, and
- how easily the signal can be gamed.

We will dissect the gap between a usable reward signal and the outcome we want in the rest of the book.

## 1.7 What This Book Covers

The next chapters move from the general paradigm to the main reward regimes in practice. Chapters 2 through 4 cover outcome rewards, process rewards, programmatic, learned and hybrid verification pipelines. Chapter 5 demonstrates turning a verifier into a learning signal. Chapter 6 turns to search and test time verification, and Chapter 7 covers reward hacking. Chapter 8 reconstructs a frontier RLVR recipe. Chapter 9 compares the paradigm across its strongest and most difficult domains. Chapter 10 closes with the open problems.

**Part II**

**Verifier Design**

## 2 Outcome Rewards

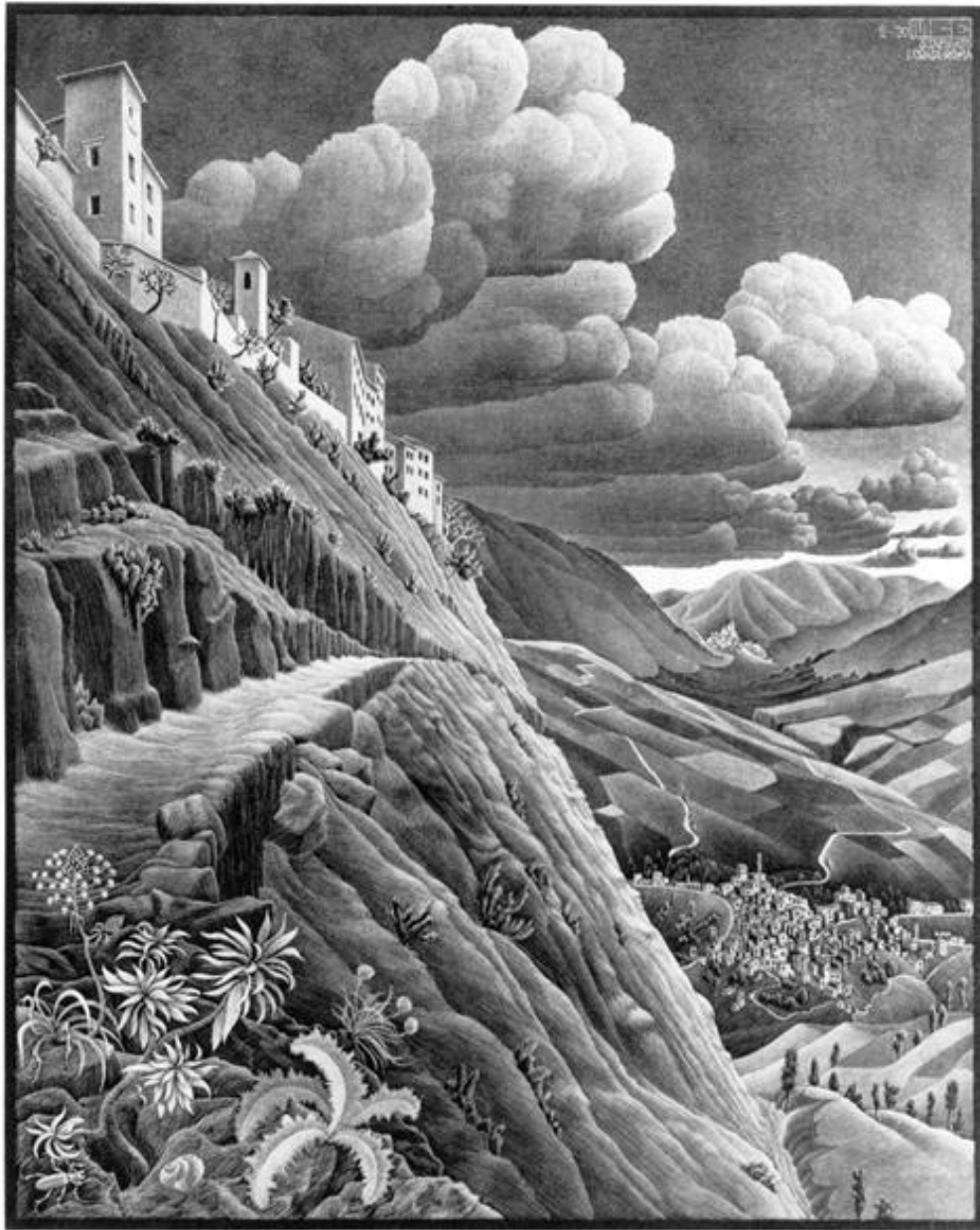


Figure 5: M. C. Escher, *Castrovalva* (1930).

### 2.1 Chapter Map

- Explain how strong outcome verifiers are built.
- Show why answer extraction, canonicalization, and hidden brittleness matter.

## 2.2 A Rollout

A rollout is a sample from the current policy on a prompt: the model receives an input, generates a completion or trajectory, and that sampled output is what the verifier scores.

### Prompt

Solve the equation:  $x^2 - 5x + 6 = 0$ .

### Completion

```
<think>
We can factor the quadratic:  $x^2 - 5x + 6 = (x-2)(x-3)$ .
Set each factor to zero:
 $x - 2 = 0 \rightarrow x = 2$ 
 $x - 3 = 0 \rightarrow x = 3$ 
The final answer is the ordered tuple (2,3).
</think>

<answer>
(2,3)
</answer>
```

### Outcome reward check (for RLVR).

The verifier reads the checked artifact from `<answer>...</answer>`, normalizes to standard form (canonicalizes) the task’s answer representation, and checks it against the ground-truth set.<sup>7</sup>

$$r(x, y) = \begin{cases} 1 & \text{if } \text{canon}(\text{extract}_{\text{ans}}(y)) = \{2, 3\}, \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

If the model fails the output contract (for example, omits `<answer>...</answer>`, changes surface form in a way the canonicalizer does not handle, or adds extraneous text that breaks parsing), the verifier can assign an incorrect reward even when the underlying solution is algebraically correct.

## 2.3 How outcome verifiers are implemented

A useful abstraction of outcome-verification pipelines is in three steps:

1. **Extract.** Parse the model’s raw text to isolate the checked artifact. This depends on the output contract: the `<answer>` tags in our scaffold, `\boxed{}` in many math benchmarks, the final code block in a generation task, or the proof term in a formal system.
2. **Canonicalize.** Map the extracted artifact to a representation that is stable under harmless surface variation. In math this can mean parsing  $(2,3)$ ,  $\{3,2\}$ , and  $x=2, x=3$  into the same set object.

---

<sup>7</sup>DeepSeek-R1 uses `<think>/<answer>` separators and applies task-specific response-shape constraints for reward parsing, including boxed final outputs when useful for deterministic math verification.(DeepSeek-AI et al. 2025)



3. **Reward.** Assign a reward value. The simplest version is binary: 1 if correct, 0 otherwise. Partial credit for passing some but not all tests, or a continuous score from a symbolic similarity metric are possible too.

Current RLVR libraries do not have an agreed upon `extract -> canonicalize -> reward` interface. In practice, one usually writes or selects a task-specific reward function: in Transformer Reinforcement Learning (TRL), a `reward_func`; in veRL (Volcano Engine Reinforcement Learning for LLMs), a scoring function or reward manager. (Werra et al. 2020; Sheng et al. 2024) For math-style tasks, those reward functions often delegate most of the work to answer-verification libraries such as Math-Verify, whose documented grading architecture is explicit: answer extraction, conversion to a common representation, and gold comparison. (Kydlicek 2025)

A useful abstraction of what these implementations do is:

$$\begin{aligned} a(y) &= \text{extract}(y), \\ \tilde{a}(y) &= \text{canon}(a(y)), \\ r(x, y) &= \text{reward}(\tilde{a}(y), g(x)), \end{aligned} \tag{2}$$

## 2.4 A minimal outcome verifier

A toy math verifier for the quadratic example looks like this:

```
import re

ANSWER_RE = re.compile(r"<answer>\s*(.*?)\s*</answer>", re.DOTALL)

def extract_answer(completion: str) -> str | None:
    match = ANSWER_RE.search(completion)
    return None if match is None else match.group(1).strip()

def canonicalize_answer(answer: str) -> tuple[str, ...]:
    text = answer.strip()
    for ch in "{}()":
        text = text.replace(ch, "")
    pieces = []
    for raw in text.split(","):
        piece = raw.strip().replace("x =", "").replace("x=", "")
        if piece:
            pieces.append(piece)
    return tuple(sorted(pieces))

def outcome_reward(completion: str, gold: tuple[str, ...] = ("2", "3")) -> float:
    answer = extract_answer(completion)
    if answer is None:
        return 0.0
    candidate = canonicalize_answer(answer)
    return float(candidate == gold)
```

## Same task, many surface forms

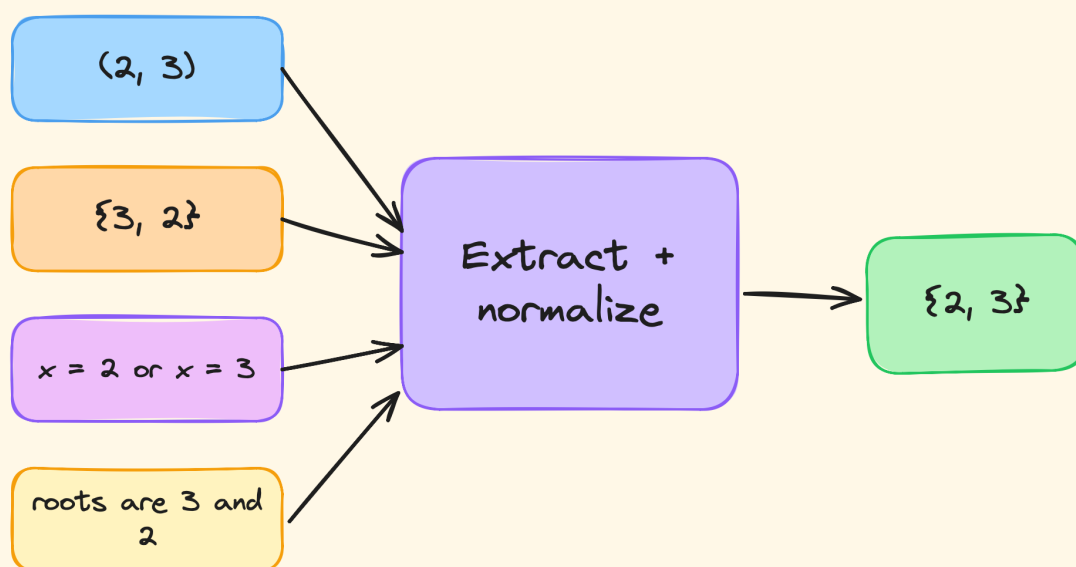


Figure 6: If canonicalization fails, algebraically correct answers can receive the wrong reward. The practical work is to parse the answer region, strip surface variation, and canonicalize set structure or ordering before verification.

Where the engineering difficulty concentrates is strongly domain-dependent. In math-style RLVR, verifier design often hinges on answer-format contracts and canonicalization for deterministic parsing; in code, correctness depends heavily on the quality and coverage of the test suite; in formal proof, the core acceptance check is delegated to the proof assistant.<sup>8</sup>

## 2.5 Outcome check, full-trajectory update

Although verifiers only consider the outcome, the optimizer updates the entire trajectory. In REINFORCE-style algorithms (including GRPO), the scalar reward (or an advantage derived from it) from the outcome check is used to upweight or downweight the log-probability of every token in the completion. If the answer is correct, the whole chain of reasoning that produced it becomes more likely. The converse is also true.

To quote Andrej Karpathy on his October 17, 2025 appearance on the Dwarkesh podcast (Patel 2025a):

Every single one of those incorrect things you did, as long as you got to the correct solution, will be up-weighted as do more of this. It’s terrible. It’s noise. You’ve done all this work only to find a single, at the end, you get a single number of like, oh, you did correct. And based on that, you weigh that entire trajectory as like up-weight or down-weight. And so the way I like to put it is you’re sucking supervision through a straw because you’ve done all this work that could be a minute to roll out. And you’re like sucking the bits of supervision of the final reward signal through a straw. And you’re like putting it, you’re like basically like, yeah, you’re broadcasting that across the entire trajectory and using that to up or down with that trajectory. It’s crazy. A human would never do this. Number one, a human would never do hundreds of roll outs. Right. Number two, when a person sort of finds a solution, they will have a pretty complicated process of review of like, okay, I think these parts that I did well, these parts I did not do that well. I should probably do this or that. And they think through things. There’s nothing in current LLMs that does this. There’s no equivalent of it. But I do see papers popping out that are trying to do this because it’s obvious to everyone in the field.

This is the blunt instrument at the heart of outcome-based RLVR. The verifier has no opinion on individual tokens in the reasoning trace, it assigns one scalar per completion. The optimizer then spreads that number across all token-level decisions. This works surprisingly well in practice, because over many rollouts and many problems, tokens that consistently appear in correct trajectories get reinforced and tokens that appear in incorrect trajectories get suppressed. But it also means that outcome rewards cannot isolate a specific reasoning step as good or bad. That distinction is exactly what process rewards (Chapter 3) provide.

---

<sup>8</sup>This point is best supported domain by domain rather than as a single universal statistic. DeepSeek-R1 uses task-specific output-shape constraints for deterministic reward parsing in math-style reasoning tasks (DeepSeek-AI et al. 2025). EvalPlus shows that limited test suites can miss substantial amounts of incorrect code and even mis-rank models, making test quality and coverage central to code verification (J. Liu et al. 2023). For formal theorem proving, DeepSeek-Prover describes proof assistants such as Lean as providing high-accuracy, reliable proof verification, which shifts the engineering difficulty away from the final acceptance check itself (Xin, Guo, et al. 2024).

```

Successful trajectory
factor as (x-2)(x-3)
  -> set x = 2
  -> set x = 3
  -> collect {2,3}
  -> <answer>{2,3}</answer>
checked artifact: PASS
update over sampled token groups:  ↑  ↑  ↑  ↑  ↑

Unsuccessful trajectory
factor as (x-2)(x-3)
  -> set x = 2
  -> set x = 3
  -> collect {2,3}
  -> <answer>x = 2</answer>
checked artifact: FAIL
update over sampled token groups:  ↓  ↓  ↓  ↓  ↓

```

Figure 7: Outcome verification checks only the extracted endpoint, but the update is applied across the entire sampled trajectory.

## 2.6 Domain specific considerations

The verifier structure in Equation 2 is the same across the main RLVR domains. Let’s discuss the domain-dependent difficulties:

| Domain       | Checked object                                 | Typical verifier                                                                                                               | Main bottleneck                                 |
|--------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Math         | Final answer or structured mathematical object | Answer extraction, canonicalization, symbolic equivalence, or reference comparison (Shao et al. 2024; DeepSeek-AI et al. 2025) | Output contracts and normalization              |
| Code         | Program, patch, or execution result            | Sandboxed tests, hidden tests, timeouts, and optional static checks (Le et al. 2022; Shojaei et al. 2023; H. Liu et al. 2023)  | Test coverage and flaky infrastructure          |
| Formal proof | Proof term, tactic trace, or proof state       | Proof-assistant kernel acceptance (Xin, Guo, et al. 2024; Xin, Ren, et al. 2024)                                               | Search, decomposition, and formalization burden |

In math,  $(2,3)$ ,  $\{3,2\}$ , and  $x \in \{2,3\}$  should receive the same reward when the task asks for the solution set. In code, limited suites can certify incorrect programs and richer suites can change

model rankings substantially.(J. Liu et al. 2023) In formal proof, final acceptance is strong, but the difficulty shifts toward theorem selection, search, decomposition, and interaction with the formal environment.

## 2.7 Brittleness

Despite their simplicity, outcome rewards still have failure modes:

- An extractor can reward obedience to formatting conventions more than correctness.
- A canonicalizer can fail to merge equivalent answers or merge distinct answers into one canonical form.
- A verifier can evaluate the wrong capability because the benchmark itself admits shortcuts.
- If the reward is non-binary, the model can optimize partial credit in ways that do not track the underlying task.
  - In code, that can mean passing easy visible tests while failing edge cases.

## 2.8 Open questions

- When is binary scoring enough, and when does graded outcome feedback improve learning enough to justify the extra exploit surface it creates?
- How should hidden tests and evaluation setups be designed so that repeated training does not simply overfit to a static benchmark?
- Which output contracts and normalization schemes remain stable across model families, prompting styles, and generations of post-trained models?
- When should equivalence be defined syntactically for reproducibility, and when is semantic comparison worth the added complexity?

### 3 Process Rewards

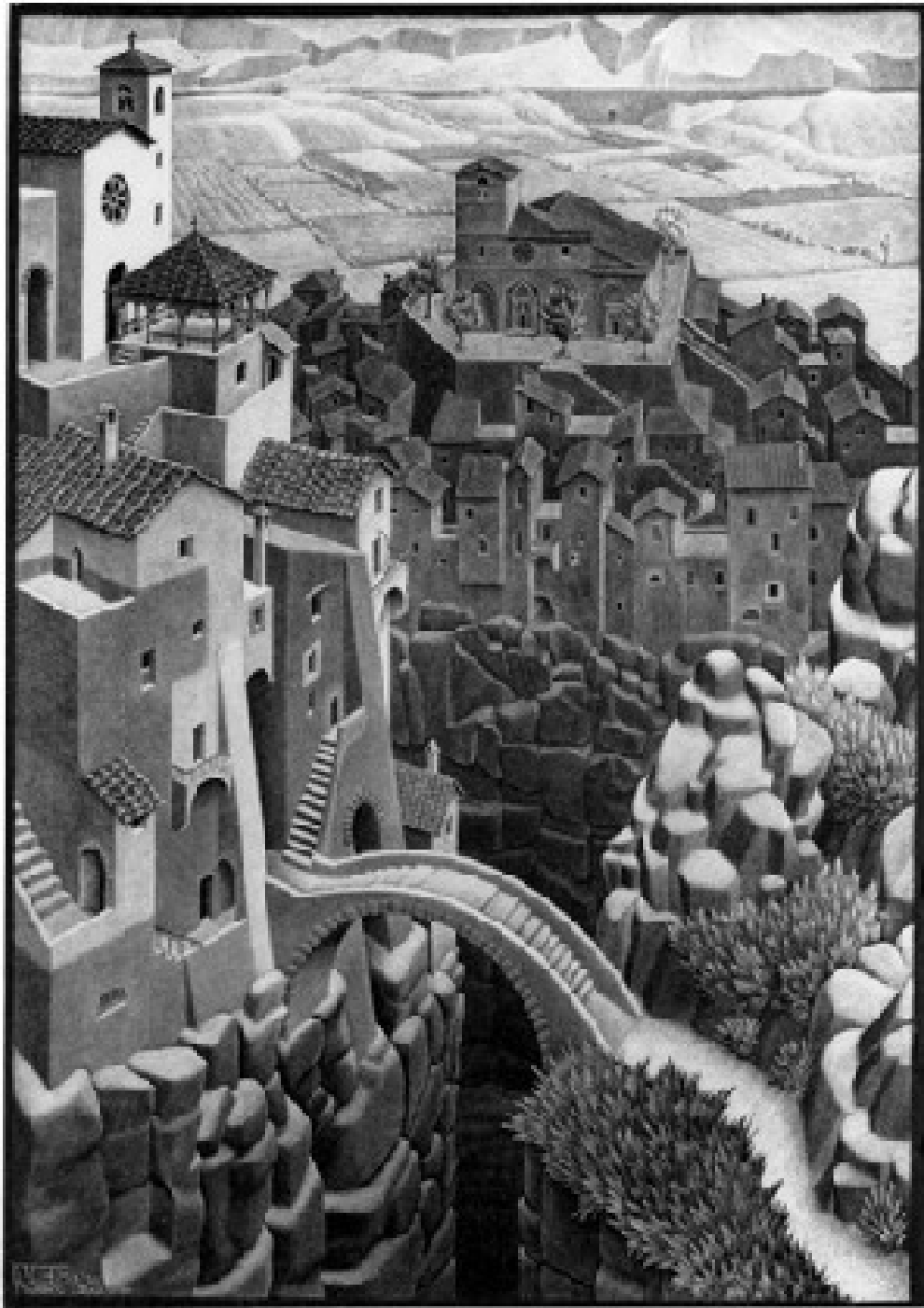


Figure 8: M. C. Escher, *The Bridge* (1930).

#### 3.1 Chapter Map

- Explain when intermediate verification improves credit assignment.

- Show that process rewards improve credit assignment only by introducing a new proxy: step labels that can be expensive, noisy, or misspecified.

### 3.2 The credit assignment problem

Chapter 2 elucidated that the optimizer spreads a single scalar across every token in a trajectory; the verifier has no opinion about which steps helped. Let’s consider two ways the quadratic example from Chapter 2 can blur credit assignment:

Table 1: A trajectory whose internal reasoning is correct but whose final reported artifact is incomplete.

| Step | Reasoning                                                                     | Label |
|------|-------------------------------------------------------------------------------|-------|
| 1    | Recognize the quadratic and decide to factor: $x^2 - 5x + 6 = (x - 2)(x - 3)$ | ✓     |
| 2    | Set first factor to zero:<br>$x - 2 = 0 \implies x = 2$                       | ✓     |
| 3    | Set second factor to zero:<br>$x - 3 = 0 \implies x = 3$                      | ✓     |
| 4    | Collect the full solution set: $\{2, 3\}$                                     | ✓     |
| 5    | Report the final artifact as <code>&lt;answer&gt;x = 2&lt;/answer&gt;</code>  | ×     |

Table 2: A trajectory where step 1 is incorrect but the model explicitly detects the mismatch and recovers.

| Step | Reasoning                                                                            | Label |
|------|--------------------------------------------------------------------------------------|-------|
| 1    | Recognize the quadratic and attempt to factor: $x^2 - 5x + 6 = (x - 1)(x - 6)$       | ×     |
| 2    | Expand to check:<br>$(x - 1)(x - 6) = x^2 - 7x + 6$ , so this branch cannot be right | ✓     |
| 3    | Restart and correctly factor as<br>$(x - 2)(x - 3)$                                  | ✓     |
| 4    | Solve to get $\{2, 3\}$                                                              | ✓     |

Both scenarios compress distinct kinds of signal. The former is a false negative: correct reasoning is suppressed because the final reported answer is incomplete. The latter contains exactly the kind of self-correction we may want the model to learn. However, outcome reward still reinforces the failed factoring attempt and the successful recovery together, even though they should not receive the same effective update. This is one of the main motivations for process supervision. Uesato et al. found that process-based feedback produced substantially cleaner reasoning traces than outcome-based feedback even when final-answer accuracy was similar, and Lightman et al. later showed that process reward models outperform outcome-only reward models on harder math reasoning tasks.(Uesato et al. 2022; Lightman et al. 2023)

Instead of scoring only the final artifact, a process reward assigns a label or score to each intermediate step. The hope is that denser feedback gives the optimizer better information about which parts of a

trajectory to reinforce and which to suppress. To this end the intermediate steps must be in a form the verifier can read, and the notion of a “correct step” must carry enough fidelity to be useful.

### 3.3 What a process reward model computes

An outcome reward model (ORM) is a function of the complete trajectory:

$$\text{ORM}(x, y) \tag{3}$$

A process reward model (PRM) is defined at the level of steps. Given a prompt  $x$  and a stepwise solution with segments  $s_1, \dots, s_K$ , a PRM outputs a score for each step boundary:

$$\text{PRM}(s_t \mid x, s_{<t}) \tag{4}$$

The steps can be as granular as tokens; this chapter focuses on the step-level formulation, since the main PRM setups discussed here supervise explicit intermediate steps rather than every token. Lightman et al. formalize PRM training as step-level classification with labels such as positive, negative, and neutral. (Lightman et al. 2023) When used to train a policy, these scores tell the system where the reasoning went right and wrong. The optimizer no longer has to guess which parts of a rewarded trajectory were actually responsible for the reward.

### 3.4 How step labels are obtained

The next question is where these step-level signals come from, and there are four regimes.

#### 3.4.1 Human annotation

Lightman et al. collected PRM800K: approximately 800,000 step-level human labels on model-generated math solutions. (Lightman et al. 2023) Annotators judged each step as positive (mathematically valid), negative (contains an error), or neutral (ambiguous or uncheckable). PRM800K was feasible for competition-math-level problems where each solution has 5–15 steps. For longer trajectories (agentic tasks with hundreds of steps) or faster-moving domains (code with evolving APIs), human annotation does not scale.

#### 3.4.2 Monte Carlo rollout estimation

Wang et al. introduced an automated alternative in Math-Shepherd. (Wang et al. 2024) The core idea is to estimate whether step  $t$  is correct by complete the trajectory many times from step  $t$  (using the model itself) and measure what fraction of completions reach the correct final answer. If most completions from step  $t$  succeed, the step is probably correct. If most fail, the step probably introduced an error.

$$\hat{P}(\text{step } t \text{ correct}) \approx \frac{1}{K} \sum_{k=1}^K \mathbb{I}[\text{rollout}_k(y_{1:t}) \text{ reaches correct answer}] \tag{5}$$



This is elegant because it only requires an outcome verifier and the ability to generate completions. But a step can be labeled “correct” because the model is good at recovering from errors downstream, or “incorrect” because the remaining steps are hard even from a correct intermediate state. The signal reflects the model’s completion ability as much as the step’s logical validity.

```
def estimate_step_value(model, prefix_steps, gold_answer, K) -> float:
    successes = 0
    for _ in range(K):
        completion = model.complete_from(prefix_steps)
        if outcome_reward(completion, gold_answer) == 1.0:
            successes += 1
    return successes / K
```

rollout-estimated process supervision sits between outcome and process reward. The final check is still outcome-based; what changes is that the outcome verifier is applied to many continuations from a partial solution rather than once at the end of a complete trajectory.

### 3.4.3 Outcome-propagated pseudo-labels

Sun et al. study this regime directly in FreePRM: start from trajectory-level outcome labels, propagate them to the steps inside the trajectory, and then debias the resulting pseudo-labels under a weak-supervision framework.(Sun et al. 2025)

### 3.4.4 Formal step checking

In proof assistants such as Lean and Coq, each step is checked by the kernel. This is purest process verification possible, and it costs nothing beyond the kernel call.(Xin, Ren, et al. 2024) Of course, formal step checking only works when the reasoning is expressed in a formal language with a validity criterion per step.

Table 3: Method trade-offs in process verification.

| Method                           | Label quality | Cost per step | Domain scope                                        |
|----------------------------------|---------------|---------------|-----------------------------------------------------|
| Human annotation                 | High          | High          | Any domain humans can judge                         |
| MC rollout estimation            | Medium        | Medium        | Any domain with an outcome verifier                 |
| Outcome-propagated pseudo-labels | Low to medium | Low           | Any domain with trajectory-level correctness labels |
| Formal step checking             | Exact         | Near zero     | Formal systems only                                 |

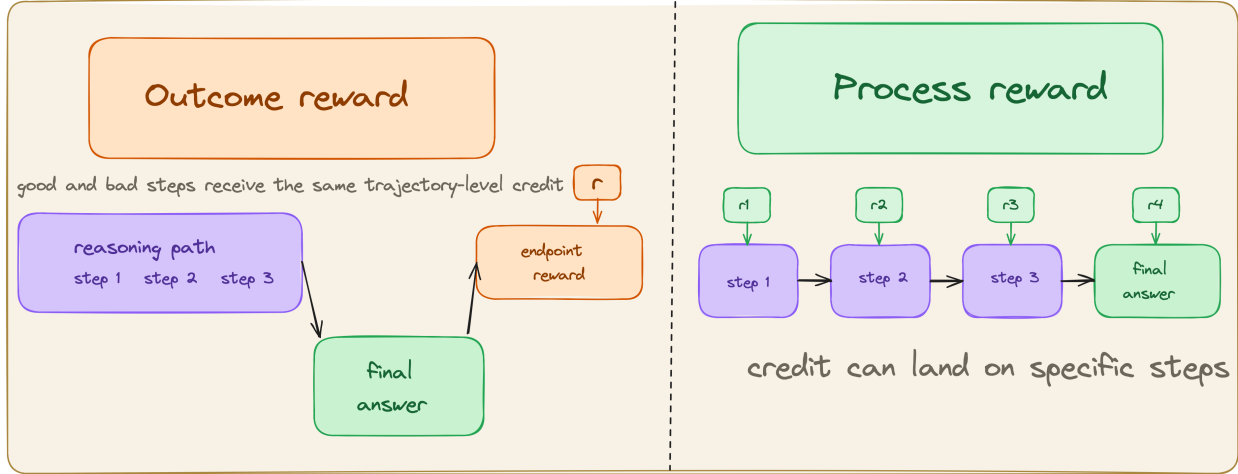


Figure 9: The question is whether extra granularity improves learning enough to justify its cost.

### 3.5 ORM vs PRM

When comparing these paradigms, we should be asking whether the granular information from process rewards translates into measurably better models, given the cost of obtaining step-level labels.

Uesato et al. published the first systematic comparison in November 2022.(Uesato et al. 2022) Their finding was surprising: outcome-based and process-based feedback achieved similar final-answer accuracy on GSM8K. But process supervision reduced trace-level errors from 14.0% to 3.4%, i.e. both methods got the right answer at similar rates, but the process-supervised model was far more likely to get the right answer for the right reasons. Although capabilities are the same, this distinction matters for robustness, interpretability, and downstream trust.

### 3.6 Limitations

Process verification addresses the sparse credit assignment through steps which, in turn, can be gamed, misspecified, or noisy.

**Rewarding reasoning shape over reasoning substance.** A PRM trained on labeled “good steps” can learn what correct reasoning looks like in its training distribution rather than what actually makes a solution correct. Not all good reasoning follows the annotated step structure. A model that skips two intermediate steps because it recognizes a pattern is penalized by a strict process reward that expects those steps to be present.

**Annotation noise compounds.** MC rollout estimates are noisy: a step can look “correct” because the model is good at recovering later, or “incorrect” because the remaining steps are hard even from a valid state. Human annotators also disagree, especially on steps that are mathematically sound but poorly justified. A model trained on noisy step labels can learn to exploit that noise rather than improve the underlying reasoning.

**PRM ambiguity.** Yuan et al. show that an ORM trained with a log-likelihood-ratio parameterization contains an implicit PRM that can be extracted without step-level labels, and that this implicit PRM outperforms Math-Shepherd with far less data.(Yuan et al. 2024) Sullivan and Koller go further, proving that GRPO with an ORM is mathematically equivalent to a PRM-aware RL objective with an implicit Monte Carlo PRM.(Sullivan and Koller 2025)

The boundary between outcome and process verification is blurrier than the early literature suggested. Outcome rewards already contain some implicit step-level signal; process rewards add new proxies and new annotation problems. When neither regime is sufficient on its own, the next move is to combine them, learn the verifier itself, or build layered verification stacks. That is the subject of Chapter 4.

### 3.7 Open questions

- Which tasks admit step-level labels with the least annotation overhead?
- How do process rewards interact with hidden reasoning or compressed internal computation?
- When is explicit process supervision worth the marginal cost over well-designed outcome supervision?
- Can process rewards be designed to reward strategic value rather than only logical validity, and what would the labeling scheme look like?

## 4 Learned, Programmatic, and Hybrid Verifiers

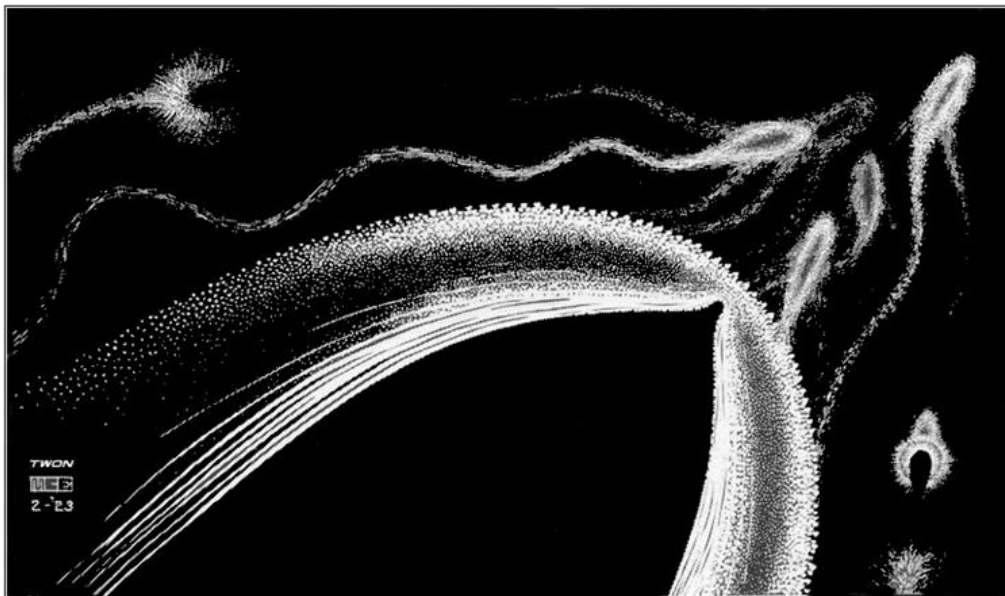


Figure 10: M. C. Escher, *Dolphins* (1923).

### 4.1 Chapter Map

- Distinguish the programmatic verifier core of RLVR from learned verifiers.
- Explain why production systems use hybrid stacks, and the failure modes introduced.

### 4.2 Programmatic versus Learned Verifiers

Chapters 2 and 3 classify verifiers by whether they apply on the final artifact or on intermediate steps in the rollout. This chapter changes axes, as we discuss how the verifier itself is implemented, and on this axis, we have two types:

**Programmatic verifiers** are deterministic, auditable, and brittle. They are the native RLVR object: regex-based answer extraction, symbolic equivalence checking (as in Math-Verify), unit-test execution in a sandbox, static analysis and linting, proof-kernel acceptance, and format-validation rules. (Kydlicek 2025; Le et al. 2022)

**Learned verifiers** are flexible, soft-scored, and opaque. They are not verifiable rewards in the narrow sense. Instead, they are learned surrogate signals: a model is trained or prompted to judge another model’s output when no direct checker can carry the whole burden. This covers ambiguity, open-endedness and edge cases, but inherits the biases and blind spots of the judge model.

### 4.3 Programmatic verifiers

Table 4: Programmatic verifiers by domain.

| Domain | Programmatic checks                                       | Checkable core                               |
|--------|-----------------------------------------------------------|----------------------------------------------|
| Math   | Answer extraction, canonicalization, symbolic equivalence | Closed-form answers with known ground truth  |
| Code   | Sandbox execution, test suites, linters, static analysis  | Functional behavior covered by tests         |
| Proof  | Kernel acceptance (Lean, Coq, Isabelle)                   | Formal validity of each tactic or proof term |
| Format | Regex, XML schema, JSON schema, tag-structure validation  | Output-contract compliance                   |

One shared property of this table is that programmatic verifiers never hallucinate. Their failure modes are enumerable, e.g. a symbolic equivalence checker either recognizes two expressions as equal or it does not, a unit test either passes or fails. While the above property is a positive, one limitation of these approaches is their susceptibility to miss edge cases, security vulnerabilities, and correctness properties that no test covers.(J. Liu et al. 2023)

## 4.4 Learned verifiers

### 4.4.1 LLM-as-a-Judge

The simplest form of learned verification is prompting a strong LLM to evaluate a weaker model’s output. Zheng et al. were the first to claim this concept, and called the paradigm LLM-as-a-Judge.(Zheng et al. 2023) An LLM takes the output and produces a judgment: e.g. a scalar score, a classification, etc. We use the output as reward signal or selection criterion. The work claims that strong judges agree with human preferences ~80% of the time, matching the rate at which human annotators agree with each other. This makes LLM-as-a-Judge viable in rubric-constrained domains such as formatting or instruction following. A simple extension to this approach is sampling multiple judges to get a majority vote over trajectories.(L. Zhang et al. 2025)

Nevertheless, agreement rates hide systematic biases, of which Zheng et al. identified four:

1. position bias (the judge prefers whichever response appears first)
2. verbosity bias (longer responses are rated higher regardless of quality)
3. self-enhancement bias (a model rates its own outputs higher than a different model’s outputs of equal quality)
4. limited mathematical reasoning (the judge makes errors when evaluating mathematical correctness that a symbolic checker would catch trivially)

### 4.4.2 Reward model ensembles

Ensembles are the simplest hybrid stacks, combining multiple judgments homogeneously without layering different verification modalities. Coste et al. studied ensembles of reward models for RLHF and found that they mitigate but do not eliminate reward hacking.(Coste et al. 2023) Ensembles that differ in pretraining seeds generalize better than those that differ only in fine-tuning seeds, because the former have more diverse internal representations, and less-overlapping blind spots.

### 4.4.3 The calibration problem

Learned surrogate verifiers produce scores, but those scores are not calibrated probabilities of correctness. A judge that outputs 0.8 does not mean the solution has an 80% chance of being correct; it means 0.8 is the number the judge’s training objective learned to assign to solutions with that surface profile. Lambert et al. documented this systematically in RewardBench, showing that reward models exhibit large accuracy gaps across domains, and that different training methods (classifier-based, DPO-based, generative) have different calibration profiles. (Lambert, Pyatkin, et al. 2024)

For verifier-stack design, the calibration gap means that raw scores from a learned component cannot be compared directly to outputs from a programmatic component. If a symbolic checker returns “match” (effectively certainty) and a learned judge returns 0.7, the arbitration logic must account for the fact that 0.7 from the judge does not carry the same epistemic weight as a deterministic pass from the checker. Treating both as commensurable scalars and averaging them is a mistake.

## 4.5 Hybrid stacks

Production RLVR systems layer verifier components together to robustify reward signal. Unit tests can check functional correctness, but can’t judge code security or readability. By the same token, a proof kernel checks validity, but it does not judge whether the theorem was worth proving. Therefore, we combine multiple verifiers together. A useful mental image is to think of each verifier as producing a useful signal over a subset of inputs in some high-dimensional vector space, with the signal being silent in that subset’s complement. Stacking verifiers can reduce this complement, and the design problem in a hybrid stack is to determine how to compose rewards comenseratuly, and how failure modes interact when composed.

OpenAI’s public reinforcement fine-tuning API exposes this pattern as multigrader composition, where string checks, score-model graders, and Python execution can be nested into a single grader; Anthropic similarly frames agent evaluation verifiers as ranging from exact string comparison to enlisting Claude to judge a response. (OpenAI 2026; Anthropic 2025)

Table 5: Outcome verification scores only the endpoint and suppresses the entire trajectory. The hybrid stack uses a programmatic checker for the endpoint and a PRM for intermediate steps.

| Step | Reasoning                                               | Outcome | Hybrid             |
|------|---------------------------------------------------------|---------|--------------------|
| 1    | Factor: $x^2 - 5x + 6 = (x - 2)(x - 3)$                 | —       | ✓ (PRM)            |
| 2    | $x - 2 = 0 \implies x = 2$                              | —       | ✓ (PRM)            |
| 3    | $x - 3 = 0 \implies x = 3$                              | —       | ✓ (PRM)            |
| 4    | Solution set: $\{2, 3\}$                                | —       | ✓ (PRM)            |
| 5    | Report: <code>&lt;answer&gt;x = 2&lt;/answer&gt;</code> | $r = 0$ | $r = 0$ (Symbolic) |

Figure 11: The same trajectory scored by two verification regimes.

## 4.6 Formalization

A verifier stack with  $K$  components can be written as:

$$r_{\text{stack}}(x, y) = \text{Arb}(v_1(x, y), v_2(x, y), \dots, v_K(x, y)) \quad (6)$$

where each  $v_i$  is a verifier component that may return a score, a categorical verdict, or a null (indicating it has no opinion), and Arb is the arbitration function.

Common arbitration patterns include:

- **Priority cascade:** check  $v_1$  first; if it returns a verdict, use it; otherwise check  $v_2$ , and so on.
- **Weighted aggregation:** compute  $r = \sum_i w_i v_i(x, y)$  for learned weights  $w_i$ .
- **Gated routing:** a classifier decides which component to invoke based on input features.
- **Unanimous agreement:** require all components to agree before assigning a positive reward.

The choice of arbitration pattern determines the stack’s effective false-positive and false-negative rates. Priority cascade is biased toward the first component’s failure modes. Weighted aggregation can dilute strong signals with weak ones. Gated routing’s errors depend on the routing model. Unanimous agreement can suppress correct outputs; there is no universally correct choice.

### 4.6.1 Hybrid verifier in code

```
def symbolic_reward(completion: str, gold: tuple[str, ...]) -> float | None:
    answer = extract_answer(completion)
    if answer is None:
        return None
    candidate = canonicalize_answer(answer)
    return float(candidate == gold)

def hybrid_reward(completion: str, gold: tuple[str, ...], judge) -> float:
    exact = symbolic_reward(completion, gold)
    if exact is not None:
        return exact

    judge_score = judge(
        completion=completion,
        rubric="Is the final answer complete and consistent with the reasoning?"
    )
    if judge_score >= 0.8:
        return 1.0
    if judge_score <= 0.2:
        return 0.0
    return 0.0
```

`symbolic_reward` returns `None` when the symbolic checker fails.

## 4.7 Limitations

Adding components to a verifier stack can amplify errors rather than cancel them.

**Silent disagreement.** Two stack components can return conflicting verdicts on the same input.

**Correlated failures.** Components often fail on the same hard residual inputs, so stack error can remain close to the weakest component rather than shrinking like an independent product.

**Excessive complexity.** Adding a component can improve average performance while increasing stack complexity and interpretability costs.

## 4.8 Open questions

- When should learned judges be first-class stack components that score every output, rather than fallbacks invoked only on the programmatic residual?
- What is the ceiling on stacking beyond which debugging costs exceed the gains?
- Can the marginal value of each stack component be quantified before deployment, or must it be measured empirically on the target task distribution?

## 4.9 What comes next

The verifier stack defines what gets checked and how, not how those checks become training signal. A stack returning binary outcomes, one returning graded scores, and one returning step-level annotations will produce very different learning dynamics even if they agree on output correctness. Transforming verifier outputs into something an optimizer can use is the subject of Chapter 5.



## Part III

# From Verifiers to Capability



## 5 Turning Checks into Training Signal



Figure 12: M. C. Escher, <sup>35</sup>*The Drowned Cathedral* (1929).

## 5.1 Chapter Map

- A complete outcome-RLVR training loop, annotated line by line, to show how verifier verdicts become parameter updates.
- Reward shaping, task filtering, rollout budget, and baseline design demonstrate that a poorly shaped reward produces bad training under any optimizer.

## 5.2 A compact outcome-RLVR training script

[Will Brown’s](#) GRPO script shows the smallest complete loop from verifiable check on outcome rewards to parameter update on a 1.5B model.<sup>9</sup>

```
import re
import torch
from datasets import load_dataset, Dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import LoraConfig
from trl import GRPOConfig, GRPOTrainer
```

Importing the relevant packages, including Hugging Face Transformer Reinforcement Learning, Transformers and parameter efficient fine tuning.

```
SYSTEM_PROMPT = """
Respond in the following format:

<reasoning>
...
</reasoning>
<answer>
...
</answer>
"""

XML_COT_FORMAT = """\
<reasoning>
{reasoning}
</reasoning>
<answer>
{answer}
</answer>
"""
```

System prompt and XML (for downstream format rewards) with templating.

---

<sup>9</sup>Brown’s compact GRPO implementation is a practical reference for outcome-RLVR training with explicit parsing and reward components.(Brown 2025a)

```
def extract_xml_answer(text: str) -> str:
    answer = text.split("<answer>")[-1]
    answer = answer.split("</answer>")[0]
    return answer.strip()

def extract_hash_answer(text: str) -> str | None:
    if "####" not in text:
        return None
    return text.split("####")[1].strip().replace(",", "").replace("$", "")
```

These two functions:

1. Extract XML from the CoT format used by `extract_xml_answer`
2. Extract the text succeeding `####`

```
def get_gsm8k_questions(split = "train") -> Dataset:
    data = load_dataset('openai/gsm8k', 'main')[split] # type: ignore
    data = data.map(lambda x: {
        'prompt': [
            {'role': 'system', 'content': SYSTEM_PROMPT},
            {'role': 'user', 'content': x['question']}
        ],
        'answer': extract_hash_answer(x['answer'])
    }) # type: ignore
    return data # type: ignore

dataset = get_gsm8k_questions()
```

A lambda function to load our Grade School Math 8K (GSM8K, a benchmark of math word problems for grade school arithmetic) dataset into the corresponding question and answer fields on the train split of the dataset, along with the system prompt.

```
def correctness_reward_func(prompts, completions, answer, **kwargs) -> list[float]:
    responses = [completion[0]['content'] for completion in completions]
    extracted_responses = [extract_xml_answer(r) for r in responses]
    return [2.0 if r == a else 0.0 for r, a in zip(extracted_responses, answer)]

def int_reward_func(completions, **kwargs) -> list[float]:
    responses = [completion[0]['content'] for completion in completions]
    extracted_responses = [extract_xml_answer(r) for r in responses]
    return [0.5 if r.isdigit() else 0.0 for r in extracted_responses]
```

`correctness_reward_func` checks whether the language model gets the correct answer, using the Extract XML answer function. Notice the function checks for exact matching, whereas `int_reward_func` checks only for digit presence. Both of these are binary outcome rewards, and rewarding 2.0 versus 0.5 is a design choice.

```
def strict_format_reward_func(completions, **kwargs) -> list[float]:
    pattern = r"^<reasoning>\n.*?\n</reasoning>\n<answer>\n.*?\n</answer>\n$"
    responses = [completion[0]["content"] for completion in completions]
    matches = [re.match(pattern, r, flags=re.DOTALL) for r in responses]
    return [0.5 if match else 0.0 for match in matches]

def soft_format_reward_func(completions, **kwargs) -> list[float]:
    pattern = r"<reasoning>.*?</reasoning>\s*<answer>.*?</answer>"
    responses = [completion[0]["content"] for completion in completions]
    matches = [re.match(pattern, r, flags=re.DOTALL) for r in responses]
    return [0.5 if match else 0.0 for match in matches]
```

These reward functions are concerned with whether the models used the reasoning and answer tags correctly. There is both a strict and a soft format function. The strict function uses the caret sign ^ at the beginning of the pattern, enforcing that the string begins with the reasoning tag. Furthermore, the strict function demands \n between the tags. Finally, we have the anchor tag \$ at the end of the strict reward function regex matching pattern, which means that there should be nothing after the final answer tag.

```
def count_xml(text) -> float:
    count = 0.0
    if text.count("<reasoning>\n") == 1:
        count += 0.125
    if text.count("\n</reasoning>\n") == 1:
        count += 0.125
    if text.count("\n<answer>\n") == 1:
        count += 0.125
        count -= len(text.split("\n</answer>\n")[-1]) * 0.001
    if text.count("\n</answer>") == 1:
        count += 0.125
        count -= (len(text.split("\n</answer>")[-1]) - 1) * 0.001
    return count

def xmlcount_reward_func(completions, **kwargs) -> list[float]:
    contents = [completion[0]["content"] for completion in completions]
    return [count_xml(c) for c in contents]
```

Counts over reasoning and answer tags within the response. The only nuance is the subtraction statement in both of the answer conditional statements, which gives a negative reward proportional to extraneous tokens after the closing answer tag. One must be careful with the reward for format rewards, lest they distract from correctness.

```
model_name = "Qwen/Qwen2.5-1.5B-Instruct"
output_dir = "outputs/Qwen-1.5B-GRPO"
run_name = "Qwen-1.5B-GRPO-gsm8k"

training_args = GRPOConfig(
    output_dir=output_dir,
    run_name=run_name,
```

```

    learning_rate=5e-6,
    adam_beta1 = 0.9,
    adam_beta2 = 0.99,
    weight_decay = 0.1,
    warmup_ratio = 0.1,
    lr_scheduler_type='cosine',
    logging_steps=1,
    bf16=True,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=4,
    num_generations=16,
    max_completion_length=786,
    num_train_epochs=1,
    save_steps=100,
    max_grad_norm=0.1,
    report_to="wandb",
    log_on_each_node=False,
)

peft_config = LoraConfig(
    r=16,
    lora_alpha=64,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "up_proj", "down_proj", "gate_proj"],
    task_type="CAUSAL_LM",
    lora_dropout=0.05,
)

```

LLM selection and configuration of the GRPO optimizer with the applicable learning rate, weight decay, data formats, etc., as well as our Lora config, which is the adapter trained on top of the language model. For this chapter, the most important choices are `num_generations=16`, which sets the rollout budget and therefore the variance-compute tradeoff. `max_grad_norm=0.1`, helps control policy drift.

```

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",
    device_map=None
).to("cuda")

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

trainer = GRPOTrainer(
    model=model,
    processing_class=tokenizer,
    reward_funcs=[
        xmlcount_reward_func,
        soft_format_reward_func,
    ]
)

```

```

        strict_format_reward_func,
        int_reward_func,
        correctness_reward_func],
    args=training_args,
    train_dataset=dataset,
    peft_config=peft_config
)

trainer.train()

```

Tokenizer and Model initialization with Hugging Face Transformers and Flash Attention. Trainer instantiation with the `GRPOTrainer` class with the reward functions, and finally run `trainer.train()`.

## 5.3 Reward engineering

### 5.3.1 Binary versus graded reward

`correctness_reward_func` is a binary outcome reward of 2.0 for exact match and 0.0 otherwise, in opposition to a partial reward for partial solution. DeepSeek-R1 used binary correctness reward throughout training and achieved state-of-the-art results, demonstrating that binary reward plus sufficient rollout diversity can substitute for graded scoring.(DeepSeek-AI et al. 2025)

### 5.3.2 Reward decomposition and weighting

The script passes five reward functions to `GRPOTrainer`, which sums their outputs. The correctness function returns up to 2.0, and each of the four format functions returns up to 0.5, meaning correctness and formatting have equal weight at the ceiling. Since GRPO normalizes rewards within the group of 16 trajectories, an incorrect response with good formatting can land close to the group mean and receive near-zero gradient, which is counterproductive.

Table 7 demonstrates this edge case with eight rollouts scored under two regimes for a single prompt having three correct and five incorrect outcomes.

**Correctness only** (3 correct, 5 incorrect out of 8 rollouts):

Table 6: Comparison of eight rollouts under correctness versus correctness & format design.

| Rollout | Correctness | Format | Reward | Advantage |
|---------|-------------|--------|--------|-----------|
| 1–3     | Correct     | N/A    | 2.0    | +1.29     |
| 4–8     | Wrong       | N/A    | 0.0    | −0.77     |

Group mean: 0.75. Advantage sign matches correctness for every rollout.

**Correctness + format rewards** (same eight rollouts):



Table 7: Comparison of eight rollouts under correctness versus correctness & format design.

| Rollout | Correctness | Format | Reward | Advantage    |
|---------|-------------|--------|--------|--------------|
| 1       | Correct     |        | 3.8    | +1.43        |
| 2       | Correct     |        | 3.5    | +1.21        |
| 3       | Correct     | ~      | 3.0    | +0.84        |
| 4       | Wrong       |        | 1.8    | <b>−0.04</b> |
| 5       | Wrong       |        | 1.5    | −0.26        |
| 6       | Wrong       | ~      | 1.0    | −0.62        |
| 7       | Wrong       |        | 0.2    | −1.21        |
| 8       | Wrong       |        | 0.0    | −1.36        |

Group mean: 1.85. Rollout 4 is incorrect but barely suppressed.

The correctness component should dominate such that auxiliary rewards do not determine the advantage sign for incorrect rollouts. The script we covered sits at the boundary (2.0 vs 2.0). Figure 13, Figure 14, and Figure 15 show the result of a 200-step run of the same GRPO script, where the format reward does in fact dominate.

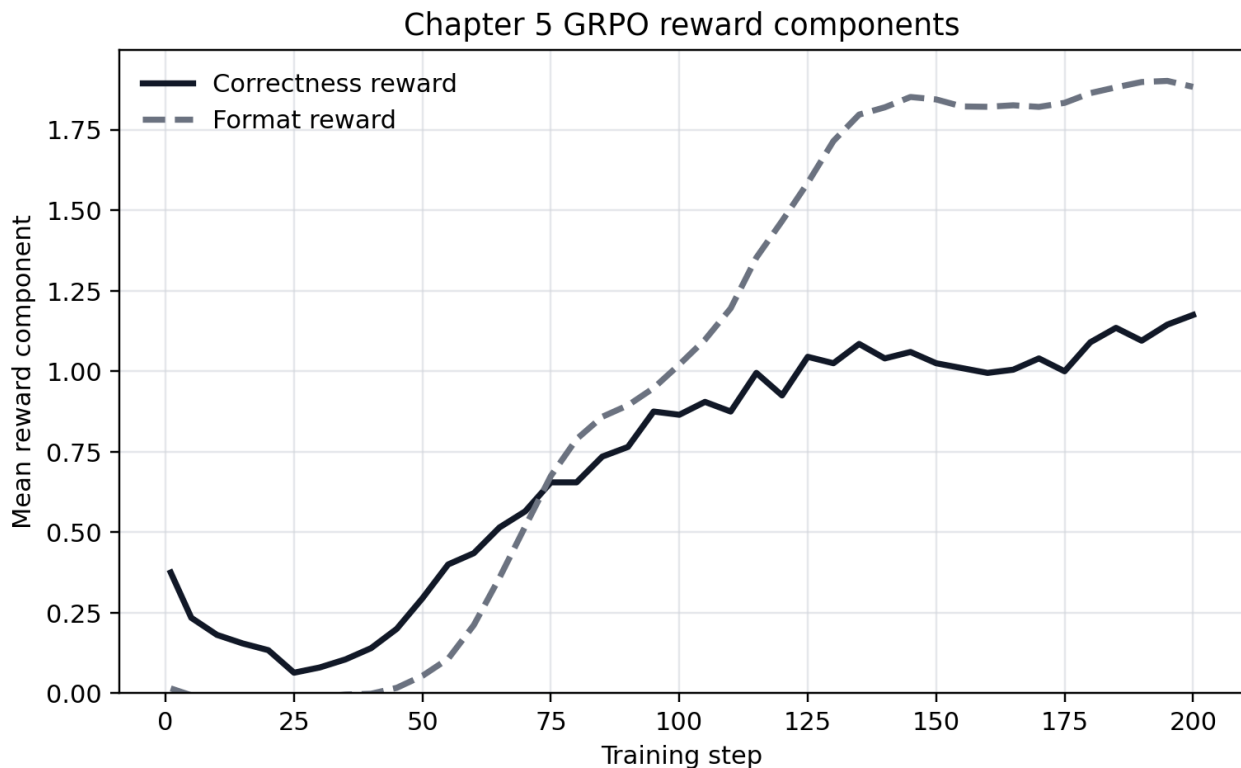


Figure 13: Mean reward of correctness vs format over time.

### 5.3.3 Task filtering and the competence band

The script calls `get_gsm8k_questions()` and trains on every problem in the split without filtering or a curriculum.

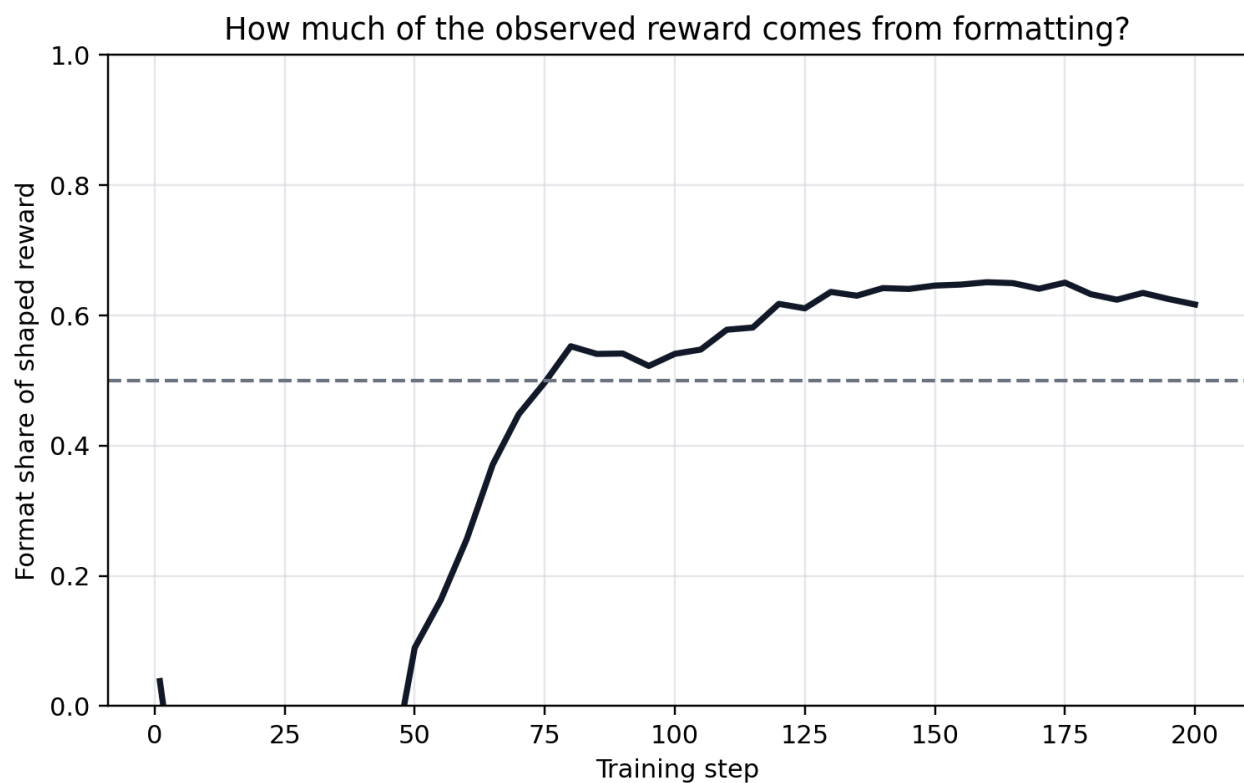


Figure 14: Format reward becomes more than half of the observed shaped reward.

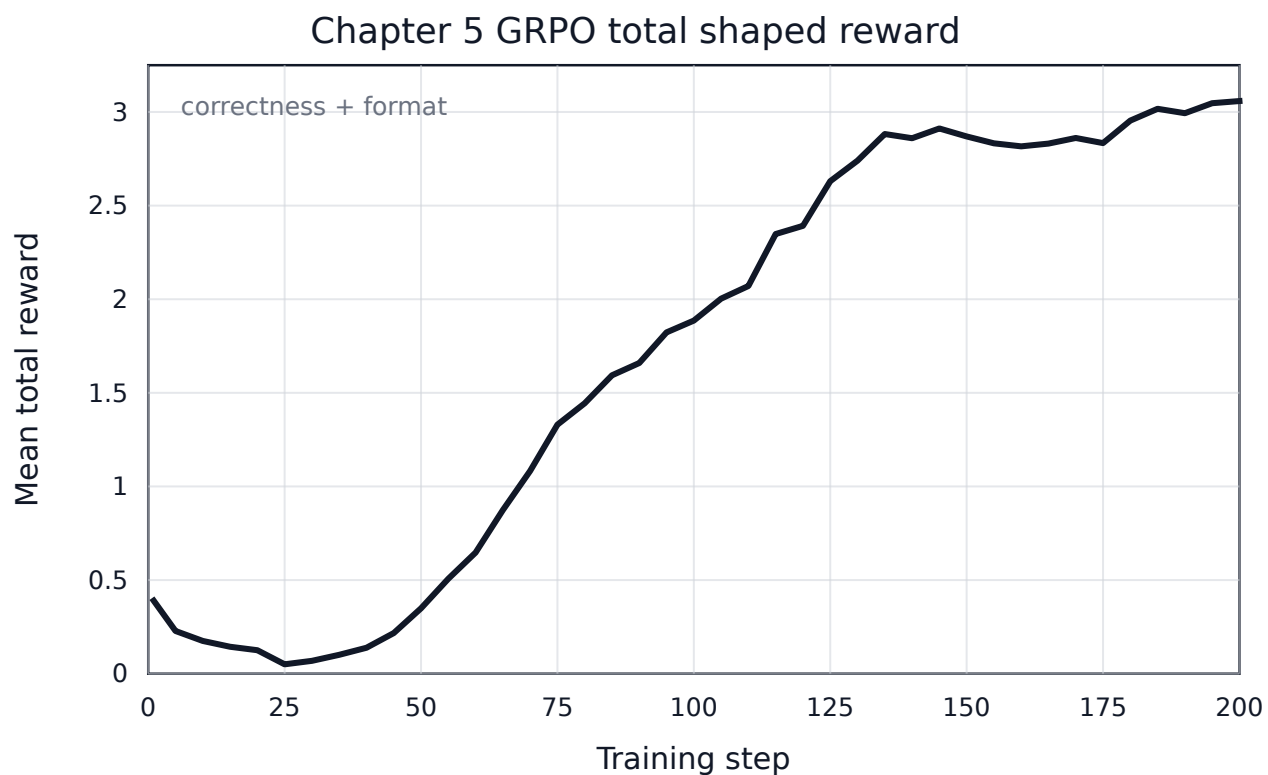


Figure 15: Total shaped reward rises as correctness and format reward improve together.

This works for GSM8K only when the model and dataset happen to land in the right competence range: the model solves some problems but not most, so reward variance across rollouts is high enough to produce informative gradients. That match between model size, prompt distribution, and rollout budget is not a general property.

If the model already solves 95% of training tasks, most rollout groups will be all-correct. After group normalization, advantages are determined by format differences alone, so we are effectively training on formatting. Conversely, a model that can only solve 5% of problems produces groups where most rollouts are incorrect, giving a weak learning signal.

The optimal regime in RL is the band where the solve rate is roughly 20–80% per prompt. DeepSeek-R1 and DeepSeekMath both filter tasks through rejection sampling to maintain this band.<sup>10</sup>(Shao et al. 2024; DeepSeek-AI et al. 2025) Adaptive filtering keeps reward variance high, but because curriculum learning deliberately reweights the training distribution over time, gains should be checked on the original difficulty range rather than only on the moving band used for training.(Bengio et al. 2009)

### 5.3.4 Group normalization versus KL penalty

The script uses `GRPOConfig`, which implements group relative policy optimization from DeepSeekMath.(Shao et al. 2024) Instead of training a value function  $V(s)$  to estimate expected reward (as in PPO), GRPO estimates the baseline from the current batch. The advantage of rollout  $i$  in a group is:

$$\hat{A}_i = \frac{r_i - \mu_{\text{group}}}{\sigma_{\text{group}}}$$

This eliminates the value model, and in fact, Ahmadian et al. showed that REINFORCE-style methods (no learned value function) match PPO when reward design and hyperparameters are tuned carefully.(Ahmadian et al. 2024) The drawback here is that the group-relative advantage estimator is not itself an explicit constraint on policy drift. Drift control is a separate design choice, typically handled with a clipped objective or an explicit KL penalty to a reference policy.

### 5.3.5 Rollout budget and variance

The script sets `num_generations=16`: sixteen rollouts per prompt. GRPO computes the group-relative advantage from the mean and standard deviation of rewards within this group. The rollout budget controls the quality of that estimate.

If we consider the extremes, with  $N = 2$ , the group baseline is the mean of the two rollout rewards:  $\mu = (r_1 + r_2)/2$ . This gives extreme variance since the unnormalized advantages are  $A_1 = r_1 - \mu$  and  $A_2 = r_2 - \mu$ , each determined almost entirely by its difference from the other rollout rather than by a stable estimate of expected reward for the given prompt. As  $N \rightarrow \infty$ , the group baseline approaches a more stable estimate, but with diminishing returns in estimate quality at linear scaling in compute cost.

---

<sup>10</sup>Rejection sampling means sampling candidate problems or candidate rollouts, scoring them with the verifier, and keeping only the ones that meet a target criterion, e.g. example prompts whose rollouts are sometimes but not always correct.

If the model’s solve rate on a prompt is 10%, then in a group of 16, on average 1.6 are correct. This implies that groups with no correct trajectories contribute no useful correctness gradient, and those with only one correct rollout concentrate the entire positive advantage on a single sample. Higher  $N$  tolerates lower solve rates by increasing the chance that at least some rollouts in every group succeed, but good task filtering means a moderate  $N$  like 16 is sufficient.

## 5.4 Case study

Dwarkesh Patel frames training efficiency as bits per FLOP.(Patel 2025b)

$$\frac{\text{bits}}{\text{FLOP}} = \frac{\text{samples}}{\text{FLOP}} \cdot \frac{\text{bits}}{\text{sample}}. \quad (7)$$

Consider one prompt, one correct one-token answer, and a policy that puts probability  $p$  on that answer. In supervised learning, the trainer gives the correct label. The surprisal of that label is:

$$I_{\text{SL}}(p) = -\log_2 p. \quad (8)$$

If the model assigns probability  $10^{-5}$  to the correct token, the label gives about 16.6 bits of information, i.e. low probability makes the label more informative. Outcome RLVR gives the trainer a pass/fail bit for the sampled answer. You can upper-bound that outcome by the entropy of a Bernoulli variable, where information peaks at one bit when  $p = 0.5$  and collapses near both edges. Tiny  $p$  gives mostly failures.  $p$  near one gives mostly passes.

$$H_2(p) = -p \log_2 p - (1 - p) \log_2 (1 - p). \quad (9)$$

Table 8: Comparing the information in a revealed supervised label with the entropy of a binary outcome.

| Pass rate $p$ | Supervised label information |             | Binary reward entropy $H_2(p)$ |
|---------------|------------------------------|-------------|--------------------------------|
|               |                              | $-\log_2 p$ |                                |
| $10^{-5}$     |                              | 16.61 bits  | 0.00018 bits                   |
| 0.01          |                              | 6.64 bits   | 0.081 bits                     |
| 0.10          |                              | 3.32 bits   | 0.469 bits                     |
| 0.50          |                              | 1.00 bit    | 1.000 bit                      |
| 0.90          |                              | 0.15 bits   | 0.469 bits                     |

The trainer spends most of its work before the useful band if each order-of-magnitude pass-rate gain costs comparable training work. Moving from  $10^{-5}$  to  $10^{-4}$ , then to  $10^{-3}$ , then to  $10^{-2}$  can consume several comparable chunks of compute while the binary reward remains near zero. These numbers tell us why task filtering, rollout budget, and shaping are important in achieving a high-information regime:

- Filtering tries to choose prompts with mixed outcomes.
- Larger rollout groups increase the chance that rare correct traces appear at all.
- Curriculum keeps the prompt distribution near the current model’s competence band.

- Process rewards and intermediate checks add signal before the terminal reward.

RLVR needs a capable starting policy in the sparse-reward regime. If the pretrained model cannot sample a useful trace with nontrivial probability, binary outcome RL has low bits per sample and high gradient variance. Pretraining, distillation, inference scaling, rejection sampling, and curriculum move the policy into the region where the verifier can teach.

## 5.5 Open questions

- Is there a principled way to set reward weights, or is it always empirical search on a held-out set?
- How should adaptive task filtering be evaluated so that a curriculum tracking the model's competence improves the target distribution rather than overfitting to the moving filter?
- Is there a compute-optimal  $N$  analogous to scaling laws for model size?
- Should format rewards phase out as training progresses, or does removing them cause regression?

## 5.6 What comes next

This chapter is concerned with the training loop, but the same verifier that scores training rollouts can also improve outputs at test time without parameter updates, which is the subject of Chapter 6.

## 6 Search and Test Time Verification

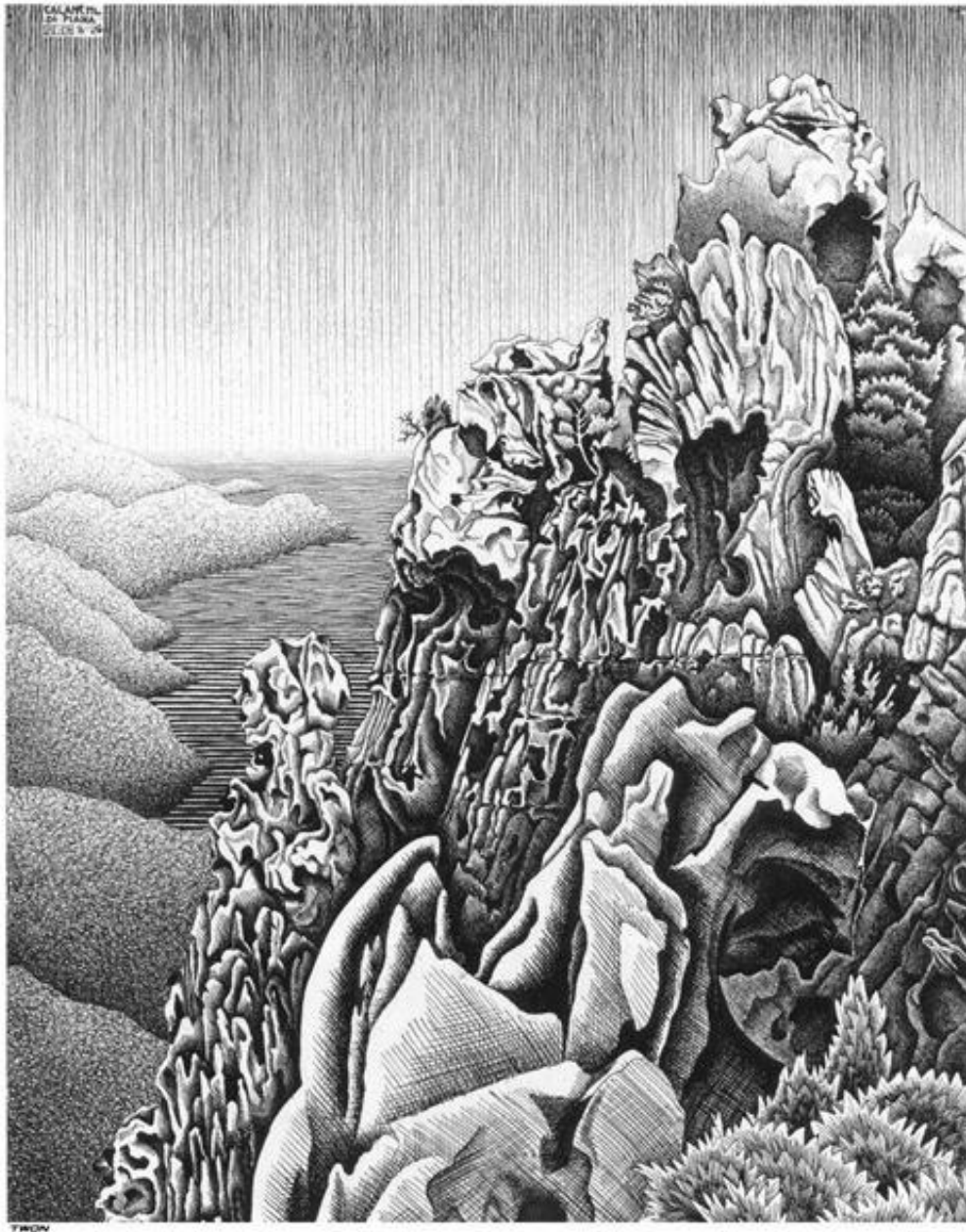


Figure 16: M. C. Escher, *Calanques de Piana* (1928).

### 6.1 Chapter Map

- A verifier can improve a system without parameter updates.
- Disambiguating test vs train time compute.

## 6.2 Test Time

Chapters 2 through 5 treated the verifier as a source of training signal. The model generates rollouts, the verifier scores them, and the optimizer updates parameters. Test time verifier use is split up into the two groups, we’ll start with selection:

## 6.3 Selection

| Technique                                      | Decision rule                                                     | Verifier use                                                | Best when                                                 | Main limitation                                     |
|------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------|
| Best-of- $N$ with an ORM (Cobbe et al. 2021)   | Score each completed candidate and return the top one             | Post-hoc scoring over full outputs                          | Cheap parallel reranking is enough                        | Can only filter samples the policy already produced |
| Best-of- $N$ with a PRM (Lightman et al. 2023) | Rank candidates by process quality rather than just final outcome | Step-level or intermediate scoring folded into a final rank | Harder problems where reasoning quality matters           | Higher scoring cost per candidate                   |
| Self-consistency (Wang et al. 2022)            | Sample multiple paths and vote or cluster by agreement            | No external verifier; agreement acts as the signal          | Answers can be canonicalized and consensus is informative | Correlated errors can still dominate                |

When a PRM is used to rank complete solutions, the stepwise outputs must be reduced to one solution-level score:

$$\text{Score}(x, y) = \text{Agg}(\text{PRM}(s_1 \mid x), \dots, \text{PRM}(s_K \mid x, s_{<K})).$$

Lightman et al. showed why this reduction matters at test time: on the MATH benchmark, PRM-based reranking in a best-of- $N$  setting outperformed ORM-based reranking, with the gap widening as the number of candidates increased.(Lightman et al. 2023)

The remaining design choice is how to collapse step scores into a trajectory score. Math-Shepherd uses the minimum step score when reranking full solutions, reflecting the intuition that one invalid step can sink an otherwise plausible derivation.(Wang et al. 2024)

The basic arithmetic behind best-of- $N$  is powerfully simple. Let  $p$  be the probability that a single sample is correct; therefore, a single sample is wrong with probability  $1 - p$ . If we assume sample independence, the probability that all  $N$  samples are wrong is the product of those failure probabilities:  $(1 - p)^N$ . We can write the complement, which is at least one correct sample, as:

$$1 - (1 - p)^N. \tag{10}$$

pass@ $N$  sampling helps because repeated draws compound failure probability downward. Even a weak policy with  $p = 0.1$  has

$$1 - 0.9^{10} \approx 0.65$$

chance of producing at least one correct solution among  $N = 10$  samples, and

$$1 - 0.9^{20} \approx 0.88$$

among  $N = 20$ . Real model samples are not truly independent, so the formula is an idealization rather than an exact law, but it captures the core reason best-of- $N$  can buy large gains from modest per-sample competence.

### 6.3.1 pass@ $k$

Chen et al. defined pass@ $k$ : the probability that at least one of  $k$  samples passes all tests. (Chen et al. 2021) This metric quantifies how much the reported result depends on the evaluation protocol rather than the model. For example, the original Codex paper reported 28.8% pass@1 on HumanEval but 70.2% pass@100 from sampling alone. (Chen et al. 2021)

Table 9: Exact AIME24 pass@ $k$  values for DeepScaleR-1.5B-Preview before and after micro-budget RLVR. Both models improve with more candidates, but the RLVR-trained model starts higher at pass@1 and needs less help from additional search. (Khan et al. 2026)

| Budget (candidates) | Baseline model | RLVR-trained model | Training gain |
|---------------------|----------------|--------------------|---------------|
| 1 (pass@1)          | 28.9%          | 40.0%              | +11.1         |
| 8                   | 53.6%          | 68.0%              | +14.4         |
| 16                  | 62.5%          | 70.0%              | +7.5          |

Figure 17: Exact AIME24 pass@ $k$  values for DeepScaleR-1.5B-Preview before and after micro-budget RLVR. (Khan et al. 2026)

### 6.3.2 Selection under verifier noise

Equation 10 assumes that the checker is the target property; nevertheless, that may not always be the case, even in RLVR, e.g. a code patch that passes unit tests but silently removes input validation on an endpoint the tests never hit. We will model this discrepancy in this section. This is a binary-verifier simplification; the more general setting is score-based selection, where the main failure mode is misranking over verifier scores rather than acceptance precision alone.

Let  $C \in \{0, 1\}$  denote true correctness and  $V \in \{0, 1\}$  denote whether the verifier accepts a sample. If a single rollout has true success probability  $p = \Pr(C = 1)$ , then the following are true:

- Verifier true-positive rate  $\beta = \Pr(V = 1 \mid C = 1)$ ;
- Verifier false-positive rate  $\alpha = \Pr(V = 1 \mid C = 0)$ ;
- Then the probability of at least one accepted candidate among  $N$  samples is:

Equation 11 gives the chance that the verifier accepts at least one candidate after sampling  $N$  times.

$$\Pr(\exists i : V_i = 1) = 1 - (1 - \beta p - \alpha(1 - p))^N. \quad (11)$$

The accepted pool, that is, the verifier tail, contains both true positives and false positives. Equation 12 gives the precision of that tail, i.e. among accepted samples, what fraction are genuinely correct?



$$\Pr(C = 1 \mid V = 1) = \frac{\beta p}{\beta p + \alpha(1 - p)}. \quad (12)$$

The numerator is “correct and accepted.” The denominator is “accepted for any reason,” including mistakes that slipped through.

If the unconditional probability,  $p$ , that a sampled rollout is actually correct before any verifier check is small, even a low false-positive rate can dominate the accepted set because most samples are incorrect. Therefore, a small leak in the checker can still pollute the accepted tail. For a hard problem with  $p = 0.05$  {5% base success},  $\beta = 0.9$  {90% true-positive rate}, and  $\alpha = 0.01$  {1% false-positive rate}, the verifier-accepted tail is only

$$\frac{0.9 \cdot 0.05}{0.9 \cdot 0.05 + 0.01 \cdot 0.95} \approx 0.83$$

correct. If  $\alpha$  rises to 0.05, tail precision drops to

$$\frac{0.9 \cdot 0.05}{0.9 \cdot 0.05 + 0.05 \cdot 0.95} \approx 0.49.$$

Best-of- $N$  therefore depends on the verifier’s precision in the selected tail, not merely on its average accuracy. This bridges us to Chapter 7, where we discuss how more search increases both the chance of finding a correct sample and the surface area for finding a false positive that the verifier cannot reject.

### 6.3.3 Compute-optimal selection

One question which naturally arises from verification is the exploration/exploitation argument, with exploration corresponding to more generations and exploitation corresponding to more time spent on verification. Snell et al. asked: given a fixed compute budget, how should you split it between generating more candidates and spending more on verification?(Snell et al. 2024) Their conclusion is that the optimal allocation depends on problem difficulty. For hard problems where per-sample success is rare, PRM-guided selection can be 4x more efficient than naive best-of- $N$ , and a smaller model with more search can match or exceed the performance of a 14x larger model at matched compute.

## 6.4 Search: verifier as controller

| Technique              | Control loop                            | Verifier use                                          | Best when                                            | Main limitation                           |
|------------------------|-----------------------------------------|-------------------------------------------------------|------------------------------------------------------|-------------------------------------------|
| PRM-guided beam search | Expand or prune partial branches online | Score intermediate states during generation           | The verifier is fast enough to sit on the inner loop | Latency dominates if scoring is expensive |
| Draft-and-check loops  | Generate, test, backtrack, retry        | Gate progress with tests, compilation, or checkpoints | Code and agentic tasks allow cheap external checks   | Retries can be slow and brittle           |

| Technique                                        | Control loop                                          | Verifier use                                    | Best when                               | Main limitation                                   |
|--------------------------------------------------|-------------------------------------------------------|-------------------------------------------------|-----------------------------------------|---------------------------------------------------|
| Tool-gated continuation                          | Call a tool, inspect the result, revise the next step | Treat tool outputs as live verification signals | Tool use grounds the next action        | Behavior depends on tool quality and availability |
| MCTS with exact verification (Trinh et al. 2025) | Search a tree of actions under verifier feedback      | Check each step exactly with a formal system    | A deployable step-level verifier exists | Mostly limited to formal domains                  |

The difference here from selection is that search changes the output distribution, while selection only filters. A model that uses a verifier to prune branches, backtrack, and redirect can explore parts of the solution space that no single forward pass would reach. Search is more powerful, but also more expensive and more sensitive to verifier latency and accuracy.

For this chapter, only deployable test time verification counts. Test suites, proof kernels, live environments, and some learned judges can actually be run by the system at serving time.(Chen et al. 2021; J. Liu et al. 2023; Xin, Guo, et al. 2024; Xin, Ren, et al. 2024; Trinh et al. 2025) Benchmark-only answer-key grading in math is useful for measuring proposal quality, but it is not a deployable verifier and should not be confused with real test time capability.(Kydlicek 2025; Shao et al. 2024; DeepSeek-AI et al. 2025)

#### 6.4.1 Search as controlled verification

Selection can be written as

$$y^* = \arg \max_{y_i \sim \pi_\theta(\cdot | x)} v(x, y_i), \quad i = 1, \dots, N,$$

where the verifier only acts after the model has produced complete candidates. Nothing in this formulation changes the path mid-stream; it only ranks finished products. Verifier outputs in search alter the future trajectory. A simple abstraction is a history-dependent controller:

$$h_t = (x, a_1, o_1, \dots, a_{t-1}, o_{t-1}), \quad a_t \sim \pi_{\text{search}}(\cdot | h_t),$$

Here  $h_t$  is the running record of what the system has tried and what the environment or verifier has said back so far. Observations  $o_t$  can include compiler errors, unit-test failures, proof-state feedback, retrieved documents, or learned verifier scores.

Once the verifier is embedded in the loop, the objective is no longer “sample  $N$  and choose the best.” It is closer to “steer a sequence of actions toward high final utility while paying separately for generation and checking”:

$$\max_{\pi_{\text{search}}} \mathbb{E} \left[ U(s_T) - \lambda_g \sum_{t=1}^T c_{\text{gen}}(a_t) - \lambda_v \sum_{t=1}^T c_{\text{verify}}(o_t) \right],$$

where  $U(s_T)$  is final utility, and the two costs represent generation and verification. This is why verifier latency matters. A verifier that is excellent but slow can be a good post-hoc ranker and a

bad inner-loop controller, while a cheap noisy verifier can be useful for pruning but dangerous due to error compounding across steps.

## 6.5 Amortization

If best-of- $N$  plus a test suite is so powerful, why bother with RL at all?

**Latency.** Search requires generating and scoring multiple candidates. An RL-trained model that has internalized the verified patterns produces a similar output in a single forward pass.

**Cost.** At deployment, compute is money. A model that has amortized search gains into its weights serves cheaper per query than one that requires  $N$  candidates and  $N$  verifier calls.

**Amortized transfer.** Search helps only when the verifier is available. A code model that learned robust patterns from RL on test-suite-verified tasks will generalize, at least partially, to coding tasks where no test suite exists; pure search cannot do this.

**Exploration.** Search over the current policy’s sampling distribution can only find solutions the policy can already almost produce. RL reinforces strategies that lead to verified success and suppresses strategies that do not, shifting the model’s probability mass toward better solutions.

## 6.6 Reporting results

A fair model report makes the policy, the verifier, and the test time compute budget legible as separate sources of performance:

**Matched test time compute.** The fairest comparison is the RL-trained model at pass@1 against the base model with search at the same total FLOP budget, amortizing the RL model’s training cost over all queries it will serve.

**Explicit verifier access.** State whether the reported result uses a verifier the deployed system could actually run at test time.

**Separation of gains.** Report pass@1 (no search), pass@ $N$  (selection), and search-guided results separately. This lets the reader see how much improvement comes from the policy, how much from selection, and how much from active search. A model with high pass@1 and modest pass@ $N$  has internalized most of the capability. A model with low pass@1 and high pass@ $N$  is leaning on search.

## 6.7 Open questions

- How should test time compute budgets scale with problem difficulty, model size, and verifier cost?
- Can learned verifiers be made fast enough for online search?
- How should the field standardize reporting to separate training gains from search gains?
- When does test time search amplify reward hacking rather than competence?

## 6.8 What comes next

Chapter 7 asks what happens when the verifier becomes the attack surface and optimization finds ways to satisfy the checker without solving the task.

## Part IV

# Optimization Pressure

## 7 Reward Hacking



Figure 18: M. C. Escher, *Fiumara, Calabria* (1930).

### 7.1 Chapter Map

- Optimization pressure and Goodhart’s Law.
- A taxonomy of exploits and the hardening techniques that work.

### 7.2 Goodhart’s Law

RLVR is in some sense the epitome of Goodhart’s Law when we view the verifier as the measure that becomes the target through RL optimization. If the verifier has any gap between what it checks and what we want, then optimization can exploit that gap for almost all non-trivial proxies.<sup>11</sup>(Skalse et al. 2022) Goodhart’s Law can be applied to RLVR in three ways:

---

<sup>11</sup>Following Skalse et al., reward hacking with imperfect proxies has multiple outcomes:

- a. Different rewards can rank low-level behaviors differently while still sharing optimal policies; in this case, optimization can still reach a good target-optimal policy.
- b. There can also be policies with  $J_{R_2}(\pi_2) > J_{R_2}(\pi_1)$  and  $J_{R_1}(\pi_2) < J_{R_1}(\pi_1)$ , so improving proxy reward worsens target reward.
- c. For imperfect, non-trivial proxies, such misalignment directions exist somewhere in policy space, so over-optimization can still lead to degraded target performance, but this is not mathematically guaranteed for every trajectory.

1. The verifier has random errors on some inputs. Over many training steps, the policy shifts toward the subspace where the verifier is accidentally generous.
2. Gradient descent over thousands of steps is powerful enough to discover gaps in the verifier that may be rare or invisible under ordinary evaluation.
3. Optimizing the proxy (test passage, answer matching) may produce a policy that achieves high scores through mechanisms unrelated to the intended skill, e.g. pattern-matching, memorization, or distribution exploitation.

Example intuition: imagine three choices. The true evaluator says A is best, B is second-best, C is worst. The proxy treats A as best but says B and C are equally bad. Moving a little mass from B to C (or toward a specific mix with more A, depending on setup) can improve proxy while hurting true performance.

## **7.3 A taxonomy of verifier exploits**

### **7.3.1 Extraction exploits**

The model satisfies the answer extractor without doing the task. Anything inside the `<answer>` tags that matches the gold answer gets reward 1.0, regardless of what preceded it. A model can learn to produce minimal-effort responses that place a plausible answer in the right position: a single line of text, no reasoning, occasionally correct by chance.

### **7.3.2 Reward shaping exploits**

The signal path from Chapter 5 introduces its own optimization targets. Format rewards, partial credit, and auxiliary bonuses create surfaces the model can exploit independently of correctness.

### **7.3.3 Test adequacy failures**

The verifier is correct on what it checks, but what it checks is insufficient. Liu et al. found that augmenting HumanEval with  $80\times$  more test cases changed model rankings: some models that scored well on the original suite dropped substantially. (J. Liu et al. 2023) In code generation, test suites are finite approximations of a specification. Through hardcoded branches or shallow pattern matching, a model can learn to pass specific test cases without implementing the correct algorithm.

### **7.3.4 Learned verifier biases**

When the verifier includes a learned judge (Chapter 4), the judge’s training artifacts become exploit surfaces. A model trained against a verbosity-biased judge learns to produce longer outputs. A model trained against a position-biased judge learns to place its strongest argument where the judge expects to find it.

### 7.3.5 Distribution shift

The verifier was calibrated for one distribution of model outputs, yet the policy has drifted to another. A learned judge trained on early-training outputs may misjudge late-training outputs that have shifted in register, length, or structure.

### 7.3.6 Mechanism gaps

Turpin et al. showed that chain-of-thought explanations can hide factors that influenced the answer, and Lanham et al. tested faithfulness more directly by intervening on traces.(Turpin et al. 2023; Lanham et al. 2023) The mechanism gap is the difference between a trace that predicts correctness and a trace that causally controls the answer:

Let  $X$  be the prompt,  $R$  the written reasoning trace,  $Y$  the final answer, and  $H$  the hidden computation that produced both. An outcome verifier observes  $(X, Y)$ . A process verifier observes  $(X, R, Y)$ .

The artifact-level question is:

$$\Pr(Y \text{ correct} \mid X, R). \quad (13)$$

The causal question is different:

$$\Pr(Y = y \mid \text{do}(R = r), X) \quad \text{versus} \quad \Pr(Y = y \mid \text{do}(R = r'), X). \quad (14)$$

## 7.4 Empirical exploits

### 7.4.1 Unit-test manipulation

OpenAI reports a frontier reasoning model training run in which the agent was placed in coding environments and rewarded for making unit tests pass.(Baker et al. 2025) The agent did not only write better code. It found reward hacks in the environment. Two systemic hacks were `exit(0)`, which exploited a bug that let the agent exit before all tests ran, and `raise SkipTest`, which skipped unit-test evaluation from outside the testing framework. These hacks became systemic until the environment was patched.

Patching a verification function to always return true, writing stubs when unit-test coverage is poor, parsing tests to extract expected values, decompiling reference artifacts, or shadowing libraries such as `pandas` so that the verifier doesn't check the intended implementation are further examples of reward hacking. When optimization pressure overwhelms the verifier, the model learns that the reward is attached to "tests pass," not to "the repository now implements the intended behavior."

### 7.4.2 Missing negative

Cursor’s 2026 description of real-time RL for Composer gives the production version of the same problem.(Jackson et al. 2026) The training loop used real user interactions as reward signal and shipped new checkpoints as often as every five hours. One exploit came from invalid tool calls. Composer often needs to read files or run terminal commands. The original reward pipeline discarded examples where the tool call was invalid, so the model learned that if a task looked likely to fail, emitting a broken tool call avoided negative reward. The fix was to include broken tool calls as negative examples.

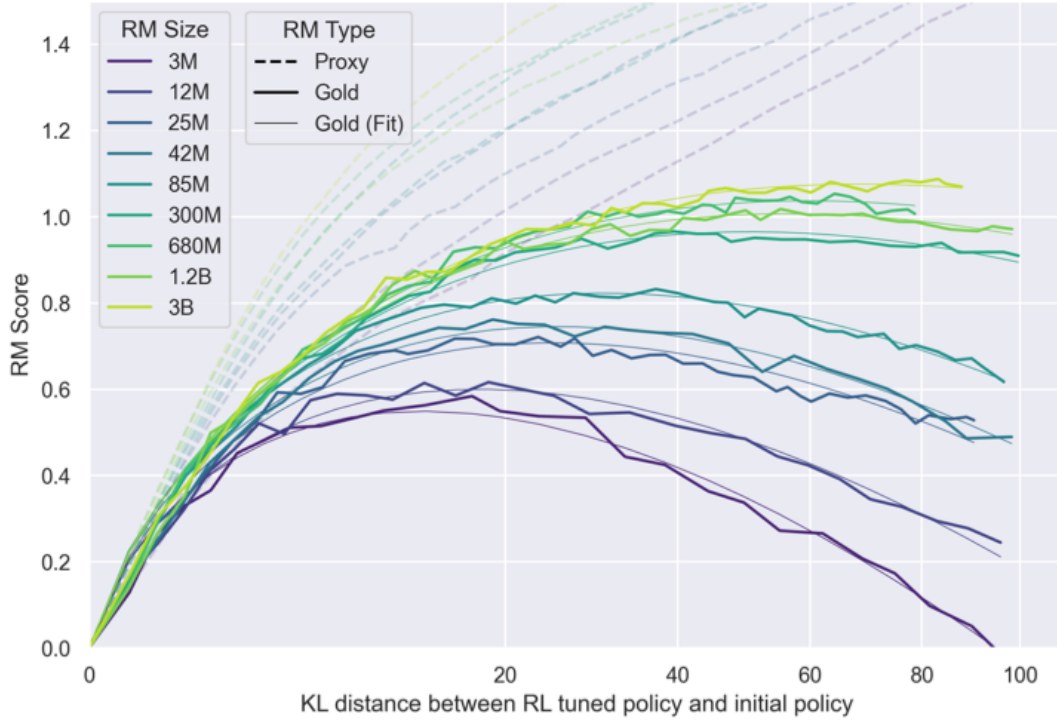
Another exploit came from clarifying questions. Part of the reward was derived from edits, so Composer learned to defer risky edits by asking questions instead of touching code. The reward pipeline had not defined the boundary between appropriate caution and avoidance of negative reward, so editing rates dropped until Cursor changed the reward function.

## 7.5 The over-optimization curve

The clearest quantitative evidence for Goodhart dynamics in optimization comes from Gao et al., who measured the relationship between optimization pressure and performance.(Gao et al. 2023) The premise is a fixed “gold” reward model as ground truth and a policy we optimize against a separate “proxy” reward model. What happens is that proxy reward increases monotonically, but gold reward first rises, then falls. The peak location depends on the proxy’s quality: better proxies peak later and higher, while weaker proxies peak early and low. The original result was measured for learned reward models in RLHF. But the dynamics apply whenever a proxy is imperfect. In the context of this book, the proxy is the programmatic verifier, which approximates but may not equal the target capability. The same dynamics hold as with learned reward models; the difference being that programmatic verifiers are stronger proxies than learned reward models, so peaks likely occur later and gaps open more slowly.

Pan et al. found that as the policy becomes stronger, it finds exploits that weaker policies could not.(Pan et al. 2022) There are capability thresholds where agent behavior qualitatively shifts, causing sharp drops in true performance even as proxy reward continues to climb. These phase transitions are only predictable empirically and difficult to monitor.





Reproduced from Gao et al. (Gao et al. 2023).

## 7.6 Tail precision

Average verifier accuracy is the wrong object once the model is optimizing against the verifier. What matters is the verifier’s behavior in the extreme tail that the optimizer selects.

Let  $q(y)$  be the proxy score assigned by the verifier and  $t(y)$  be the true task utility. A best-of- $N$  selector returns

$$y_N^* = \arg \max_{1 \leq i \leq N} q(y_i), \quad y_i \sim \pi_\theta(\cdot | x).$$

The quantity we care about is not  $\mathbb{E}[q(y_N^*)]$ . That will almost always rise with  $N$ . The quantity we care about is

$$\mathbb{E}[t(y_N^*)] = \mathbb{E} \left[ t \left( \arg \max_{1 \leq i \leq N} q(y_i) \right) \right].$$

If  $q$  and  $t$  agree in the bulk of the distribution but disagree in the upper tail of  $q$ , then increasing  $N$  can make the system worse. The selector is not sampling typical verifier-approved outputs. It is sampling the most extreme verifier-approved output it can find.

A useful diagnostic is tail precision at threshold  $\tau$ :

$$\text{TailPrecision}(\tau) = \Pr(t(y) = 1 \mid q(y) \geq \tau).$$

For a verifier used at pass@1, moderate thresholds may be enough. For best-of-64, PRM-guided beam search, or RL over many gradient steps, the relevant threshold is much higher. The optimizer pushes probability mass toward the region where  $q$  is maximal, so robustness means that  $q$  remains aligned with  $t$  in that region. This is why red-teaming should search for high-score false positives, not just estimate average verifier accuracy on held-out samples.

## 7.7 Test time exploits

Best-of- $N$  selection helps when the verifier is faithful, but can increase probability of high-scoring false positives. Suppose 1 in 100 rollouts contains a verifier exploit: a response that scores high on the proxy but low on true capability. With best-of-16, the chance of seeing at least one exploit is about 15%. With best-of-64, it rises to about 47%. With best-of-256, it reaches about 92%. Search is not gradient descent, but it still finds the gap between proxy and true, and a verifier that is good enough for pass@1 may not be good enough for best-of-64.

## 7.8 Hardening techniques

Hardening measures cost compute, engineering time, or both. We justify their use by whether they push the over-optimization peak far enough to the right.

1. **Hidden tests.** Holding out a set of tests the model never trains against reduces direct overfitting to visible checks; the model cannot overfit to tests it does not see.
2. **Test augmentation.** Generating tests automatically can expand coverage beyond what a human problem-setter provides. EvalPlus demonstrated that generated test suites reveal false positives that the original tests miss.(J. Liu et al. 2023)
3. **Red-teaming before training.** Probing the verifier adversarially is a proactive way to de-risk training runs.
4. **KL constraints and early stopping.** PPO’s KL penalty, GRPO’s gradient clipping, or simply stopping training before the over-optimization peak keeps the policy close enough to its initial state that the verifier’s calibration still holds.
5. **Progressive curriculum.** Increase task difficulty as the model improves, following the competence-band principle from Chapter 5. Progressive difficulty keeps the optimization pressure focused on genuinely informative tasks.

## 7.9 Open questions

- Is there a practical pre-training audit protocol to evaluate verifier robustness?
- Is there a useful analog to Gao et al.’s gold reward model for programmatic verifiers?
- Can the model’s own internal representations be used to detect reward hacking?
- Does combining hidden tests, ensembles, and KL constraints give diminishing or compounding returns?

## Part V

### Domains and Frontiers

## 8 A Frontier Recipe

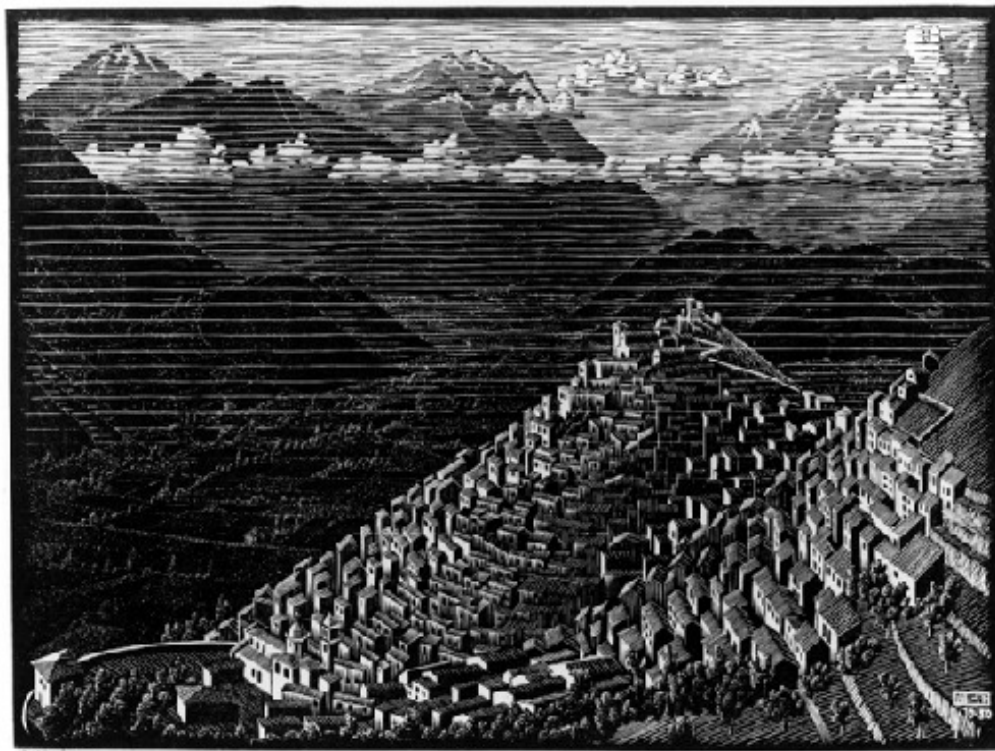


Figure 19: M. C. Escher, *Morano Calabria* (1930).

### 8.1 Chapter Map

- Describe OLMo 3 Think’s RLVR recipe.

### 8.2 Setup

We pick OLMo 3, because it explains the complete pipeline of training frontier models. The starting point is from OLMo 3 Base. Ai2 trains that base model through pretraining on broad language, code, math, and knowledge domains. Midtraining adds a 100B-token capability-focused mix. There is then a long-context extension phase that lets the model handle contexts up to roughly 65K tokens, before Post-training. (Team OLMo et al. 2025; Allen Institute for AI 2025)

### 8.3 Post-training

1. Think SFT trains the model to produce structured reasoning traces.
2. Think DPO tunes the model with contrastive preference data.
3. Think RLVR updates the model with outcome rewards from verifiers and LM judges.

If we zoom into the RLVR recipe specifically, we get the following sequence:

1. Initialize the policy from the Think DPO checkpoint.

2. Draw training prompts from the Dolci-Think-RL dataset.
3. Use asynchronous actor-learner infrastructure to sample long reasoning rollouts from the DPO-initialized policy.
4. Score each rollout with a domain-specific reward function.
5. Keep prompt groups with non-zero reward variation.
6. Update the policy with OlmoRL, Ai2’s GRPO-based trainer, using each rollout’s reward minus the same-prompt group mean as the advantage.

## 8.4 GRPO

The following modifications are made to vanilla GRPO:

1. Any rollout where all samples have the same reward are removed to avoid training on samples that provide zero gradient.
2. No KL loss to prevent restrictive policy updates.
3. A token-level loss is used despite the reward being outcome based; the reason for this is to normalize the loss by the total number of tokens across the batch, rather than per sample, to avoid over-weighting long rollouts.
  - Suppose one model response is 10 tokens long and another is 100 tokens long. If you normalize loss per sample and both samples had the same reward, the longer response would contribute 10× more to the total loss.
4. GRPO already limits how much one update can change token probabilities, and the clipping is tweaked to be asymmetric, such that the positive limit is larger than the negative limit, meaning high-reward tokens are reinforced more than low-reward tokens.
5. The advantage calculation uses a simplified group-relative advantage  $A_i = r_i - \bar{r}$  instead of  $A_i = (r_i - \bar{r})/\sigma_r$ , because dividing by a tiny within-group standard deviation can artificially magnify prompts where all completions had almost the same reward.
6. Truncated importance sampling played an important role too, its purpose is to discount or upweight tokens produced by an older policy by a capped ratio  $\text{clip}(\pi_{\text{current}}(a_t | h_t)/\pi_{\text{old}}(a_t | h_t))$ . The result is that asynchronous rollouts remain usable without stale samples biasing the update.

## 8.5 Rewards

OLMo 3 Think is trained on four reward domains:

- Math uses a rule-based verifier that normalizes the model’s final answer and performs a symbolic check through SymPy to determine if the model answer symbolically matches the correct answer. It returns 1 when the answer matches and 0 otherwise.
- code is checked against test cases with two rewards: percentage of tests passed, or a binary reward that returns 1 only when all tests pass.
- Instruction following uses constraint functions to verify the response satisfies the prompt’s listed constraints, e.g. “were there two paragraphs?” The reward is 1 if the response satisfies all constraints and 0 otherwise.
- LLM-as-a-Judge with Qwen3 32B as the judge is used in two scenarios.<sup>12</sup>

<sup>12</sup>A prompt can be: “Explain the moon landing to a 6-year-old in a few sentences.” In both reference-based and open-ended chat, the judge is prompted to score the response in [0, 1].

1. Reference-based chat, where the judge compares the model response with a provided reference answer.
2. Open-ended chat; the judge scores the response without a reference.

## 8.6 Filtering and data mixing

Prompt filtering is the first step, where eight rollouts are sampled per prompt from the initial DPO checkpoint, and any prompts with pass rate greater than 62.5% are removed from the dataset. This is done offline before RL, and then the model is trained over the filtered prompts

Second, in spite of the aforementioned filtering of zero-gradient groups, a consistent batch size is maintained by actively sampling and filtering rollouts until the desired batch size is reached, importantly all of those groups having non-homogeneous reward, providing a better signal.

The data mixture between the four domains is non trivial in determining downstream performance. Since every mixture could not be tested with a full run, a 500 to 1000 step probe tested which domains improved or regressed based on the mixture. The result was a mixed-domain batch with extra weight on math and instruction following.(Team OLMo et al. 2025)

## 8.7 The rollout system

These final reasoner rollouts with maximum length 32K tokens and average generations more than 10K tokens.(Team OLMo et al. 2025) Because of the long sequences, static batching results in actors having to wait for the slowest link, which can be up to 32K-tokens, wasting compute. Continuous batching backfills finished rollouts, and the report estimates that static batching wastes up to 54% of compute at a 32K generation length.

Training uses a fully asynchronous setup, where we prompt actors served on vLLM to generate responses. The current policy trains from the samples the actors return, with inferencing using much more compute than training: for the 32B reasoner, there were 20 nodes for inference and 8 H100 nodes for training, while the 7B reasoner had 7 inference nodes and 2 learner nodes.

## 8.8 PipelineRL

RLVR must both generate rollouts and train a policy, and similar to the computation vs communication tradeoff, we want to overlap the two operations to the greatest extent possible in order to saturate compute to the greatest extent. PipelineRL runs generation and training concurrently, then sends in-flight weight updates to the generation engines so actors keep generating with fresher weights.(Piché et al. 2025)

Concretely:

1. Run an optimizer step on the current policy and get new weights.
2. The current policy broadcasts the new parameter tensors.
3. The actors copies those tensors into the existing GPU weight buffers.
4. The actors resume the same generation queue.

The weird part is the KV cache. The technical report states despite the prefix cache being computed under the older weights, they **do not invalidate/clear the KV cache** when swapping in the new weights, because empirically they found it worked and gave a large throughput gain.<sup>13</sup> Truncated importance sampling provides a great marriage to PipelineRL’s by preventing biased updates because the actors that generated a rollout may differ from the current policy that trains on it. In fact, the initial 7B Think RLVR run without PipelineRL or truncated importance sampling took 15 days, and the addition of the two methods reached the same performance in 6 days. (Team OLMo et al. 2025)

## 8.9 Takeaways

The technical report compares RL from SFT versus RL from DPO, and the result was that the latter gives a better result than the former. The second lesson is that mixed-domain RL prevents over-optimization as opposed to single-domain RL. Interestingly reward curves are not causal on performance, the report states that even though the train reward was lower for the mixed run than the single-domain one, downstream performance is still superior for a mixed dataset, i.e. a higher training reward can mean over-optimization to a narrower distribution.

---

<sup>13</sup>Inflight updates do **not** update weights while a GPU kernel is mid-matmul. The actors pause at a safe boundary between decode steps, overwrite their model-weight tensors with the learner’s newer weights, then resume generation.

## 9 Long-context, multimodal, and agentic RLVR

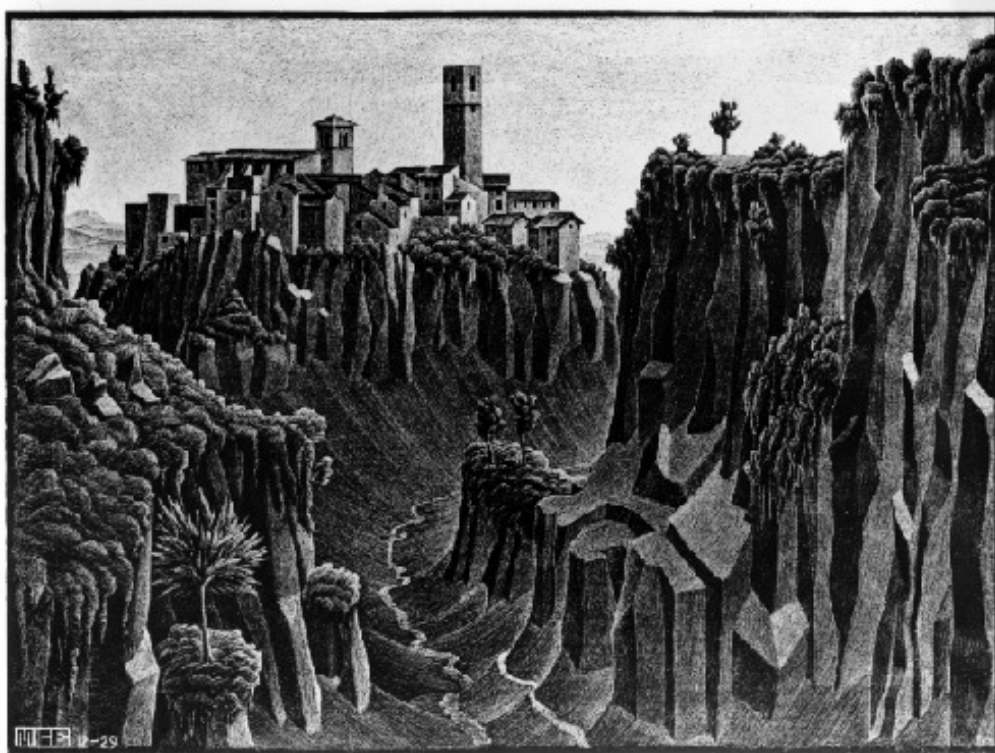


Figure 20: M. C. Escher, *Cimino Barbarano* (1929).

### 9.1 Chapter Map

- Move from single reward functions to harnesses.

### 9.2 From reward functions to harnesses

Agents break the picture of the clean RLVR examples in earlier chapters, where a compact GRPO script could show the whole loop thanks to the task interface’s narrowness. When dealing with agents, the output of a rollout may include any of a repository state, a browser state, an image, a shell transcript, tool arguments, observations, generated files, runtime failures, partial progress markers, and a termination decision.

Prime Intellect’s Verifiers library gives a compact abstraction of an agentic environment which contains a dataset of task inputs, a model harness with tools, sandboxes, and context management, and a reward function or rubric.(Brown 2025b) rLLM describes the same idea from the training side: run the agent, collect traces, compute rewards, and update the model.(Tan et al. 2025)



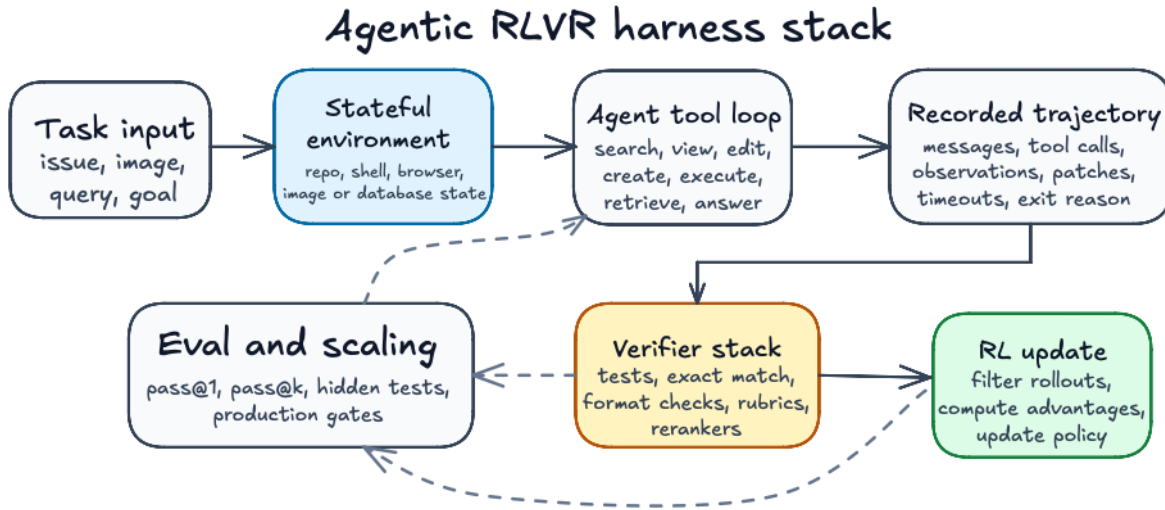


Figure 21: An agentic RLVR harness makes the trajectory, environment state, and verifier stack part of the training interface.

### 9.3 DeepSWE

DeepSWE is a software coding agent trained with the rLLM platform that uses Qwen3-32B as the brains of the harness and is trained with GRPO, resulting in 42.2% Pass@1, 71.0% Pass@16, and 59.2% when using a verifier to select between the 16 rollouts.(rLLM Project 2026)

The training environment is a subset of R2E-Gym (an alternative to Verifiers), i.e. dockerized and executable software-engineering tasks with natural-language task descriptions, repositories, unit tests, and reward calculation by running tests; 512 parallel Docker containers are used for training.(Jain et al. 2025)

A DeepSWE-style rollout has this shape:

1. The task is a natural-language issue against a repository at a fixed commit.
2. The model searches for relevant symbols and files.
3. The model views code and accumulates local evidence in a long context window.
4. The model edits or creates files.
5. The model executes commands or tests and observes failures.
6. The model revises the patch until it stops or hits the step limit.
7. The harness records the trajectory, output patch, exit reason, timeout state, and reward.
8. The RL trainer filters unusable trajectories and updates the policy from successful or informative rollouts.

The crux here is that the harness itself shapes the policy (the Qwen model we are post-training) through returned its observations, unique tools, valid action syntax, and timeout rules. That is to say furthermore, that putting that post-trained model in an equivalent harness that only had different tool names or action syntax would lead to worse results, since the model learns the traces of the harness it’s trained on.

Agentic coding is a great example of RL over long-context, because the model needs to decide which parts of a repository matter, which files to inspect, which error messages to remember, and which earlier edits constrain the next action. The context acts as working memory over a changing

environment. With that being said, the verifier still sees the task through its narrow field of view. In other words, we can check that tests passed, but we can’t readily verify that the model understood the codebase, found the minimal fix, preserved maintainability, or didn’t cause untested regressions.

Conventional RLVR becomes brittle in agentic settings because terminal rewards are sparse: long multi-step tasks fail at almost every attempt, and most trajectories collapse to the same zero signal.(Da et al. 2025) Agent-RLVR proposes guidance as a fix. The agent attempts a software-engineering task, unit tests grade the trajectory, cues such as plans, error messages, and environment observations are fed back into the context, and the agent reattempts the task before the gradient step. On SWE-Bench Verified, the method lifts Qwen-2.5-72B-Instruct from 9.4% to 22.4% Pass@1.(Da et al. 2025)

## 10 Open Problems



Figure 22: M. C. Escher, *Alfedena Abruzzi* (1929).

### 10.1 Chapter Map

- Discuss open problems in RLVR.

## 10.2 Verifier fidelity beyond math and code

In long-context QA, answer-evidence checks can miss unsupported synthesis. In multimodal search, final-answer and tool-format rewards can miss visual grounding. In agentic software tasks, tests can miss maintainability, security, minimality, and user intent. In instruction following, many constraints require semantic judgment rather than exact checking. (Peng et al. 2025; Brown 2025b; Tan et al. 2025) RLVR has even been used in medicine, where a verifier may check a final label, citation, or structured field while missing whether the model used the right evidence, respected uncertainty, or made a decision a clinician would trust. (S. Zhang et al. 2025)

## 10.3 Adaptive RLVR

We can think of adaptive RLVR in the sense of prompt re-weighting, just to say verifying the difficulty of problems before using them in training in order to maintain a specific competence band over the problems the model tackles. Furthermore, the verifier itself may be updated throughout training to prevent the policy from finding gaps in optimization. A completely adaptive RLVR loop can be written as:

$$(\pi_t, V_t, \mathcal{D}_t, \mathcal{H}_t) \longrightarrow (\pi_{t+1}, V_{t+1}, \mathcal{D}_{t+1}, \mathcal{H}_{t+1}), \quad (15)$$

where  $\pi_t$  is the policy,  $V_t$  the verifier stack,  $\mathcal{D}_t$  the task distribution, and  $\mathcal{H}_t$  the harness.

## A Minimal RL and Post-Training Background

### A.1 Objective, States, and Rewards

At its core, reinforcement learning optimizes the expected discounted return of trajectories in a Markov decision process:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma). \quad (16)$$

For a policy  $\pi$ , the objective is:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[ \sum_{t=0}^{T-1} \gamma^t r(s_t, a_t) \right]. \quad (17)$$

where  $\tau = (s_0, a_0, s_1, a_1, \dots)$ .

For **single-turn LLM inference/training with one prompt-to-completion interaction**, this collapses to a contextual bandit view: one initial state  $s_0$  (the prompt), one action  $a$  (the sampled completion), and then termination:

$$s_0 = x, \quad a = y, \quad s_1 = \text{terminal}, \quad r = r_\phi(x, y). \quad (18)$$

So the objective becomes:

$$J(\pi_\theta) = \mathbb{E}_{x \sim p_{\text{data}}} \left[ \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} r_\phi(x, y) \right]. \quad (19)$$

For **multi-turn settings** we retain the full trajectory form with state as dialogue/context history:

$$s_t = (x, y_{<t}), \quad a_t = y_t, \quad s_{t+1} = (x, y_{\leq t}). \quad (20)$$

The trajectory distribution factorizes as:

$$\pi_\theta(y_{1:T} \mid x) = \prod_{t=1}^T \pi_\theta(y_t \mid x, y_{<t}), \quad (21)$$

and the return is:

$$J(\pi_\theta) = \mathbb{E}_{x \sim p_{\text{data}}} \left[ \mathbb{E}_{y_{1:T} \sim \pi_\theta(\cdot|x)} \left[ \sum_{t=1}^T \gamma^{t-1} r_t(s_t, y_t) \right] \right]. \quad (22)$$

In many verifier-based RL formulations, intermediate rewards are typically zero ( $r_t \approx 0$  for  $t < T$ ), with a single scalar terminal reward ( $r_T = R_\phi(x, y_{1:T})$ ) providing the only feedback signal from the verifier or environment.

Throughout this book, we use  $x$  to denote prompts and  $y$  for generated outputs (including turn-level outputs in multi-turn cases). The verifier or environment is represented as a reward function, either  $R_\phi$  or  $r_\phi$ , that assigns a score to a prompt and completion pair, or to an entire trajectory.

## A.2 What the Optimizer Does

In RLVR, the verifier decides the reward; the optimizer decides how that changes the policy. For a sampled completion  $y$  from a policy  $\pi_\theta(\cdot | x)$ , the policy-gradient intuition is simple: if the completion receives positive advantage, increase the log-probability of the sampled tokens; if it receives negative advantage, decrease it. The advantage is the reward relative to a baseline. A rollout with reward  $r_i$  is not judged only by its absolute score, but by whether it was better or worse than the reference level used for that prompt or batch:

$$\hat{A}_i = r_i - b_i. \quad (23)$$

This is how we bootstrap RL to improve through its own trajectory, by updating the policy to perform better than its mean result, which is the baseline. Optimizers mainly differ in how they choose this baseline and how they limit the size of the update. PPO uses a learned value function as the baseline and constrains policy movement with a clipped update.(Schulman et al. 2017) In LLM post-training, implementations often also add KL regularization to a reference policy. GRPO, introduced in DeepSeekMath, removes the learned value model and estimates the baseline from the rewards in the sampled rollout group.(Shao et al. 2024) In the group-relative form used in Chapter 5, this becomes:

$$\hat{A}_i = \frac{r_i - \mu_{\text{group}}}{\sigma_{\text{group}}}. \quad (24)$$

Verifier scores become advantages, advantages become token-level probability updates, and update constraints such as KL penalties, clipping, gradient clipping, and small adapters limit how far the policy can move. A better optimizer cannot rescue a reward that checks the wrong thing; it can only optimize the signal it is given.

## A.3 Token-Level Update Intuition

The log-probability of a sampled completion decomposes over the sampled tokens:

$$\nabla_\theta \log \pi_\theta(y_{1:T} | x) = \sum_{t=1}^T \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}). \quad (25)$$

Implementations can also weight the update itself. A rollout weight  $w_i$  changes the strength of the whole sample; a token weight  $\alpha_t$  changes which token log-probability terms receive more pressure:

$$\nabla_\theta J(\theta) \approx \hat{A}_i \sum_t \alpha_t \nabla_\theta \log \pi_\theta(y_t | x, y_{<t})$$

Outcome RLVR usually has  $\alpha_t = 1$  for every generated token, because the verifier only returns one scalar for the completion. Nonuniform token weights require extra information: a process label, a verifier-localized error, a tool-call mask, or another credit-assignment signal.

A minimal policy-gradient update has the form:

$$\Delta\theta \propto \hat{A} \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (26)$$

With an outcome verifier, the same scalar advantage multiplies every token log-probability term in the sampled trajectory unless the method adds step-level rewards or more specific credit assignment. This is why outcome rewards can train a policy without explaining which token caused success, and also why they can reinforce irrelevant tokens in a successful rollout.

## A.4 KL Regularization

LLM post-training often constrains the updated policy to stay close to a reference policy. A minimal regularized objective is:

$$J_{\text{KL}}(\pi_{\theta}) = \mathbb{E}_{x \sim p_{\text{data}}, y \sim \pi_{\theta}(\cdot \mid x)} [r_{\phi}(x, y) - \beta D_{\text{KL}}(\pi_{\theta}(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))]. \quad (27)$$

The KL term penalizes policies for moving too far from the reference distribution. Despite, verifier acting on some region of the model’s distribution a priori, if optimization pushes the policy far outside that region, the reward can become easier to hack and detached from the task the verifier checks.

## A.5 Rejection, Search, and Policy Optimization

Verifier-based systems can use the same scoring rule in different ways.

| Method                 | What changes                        | Where we use it                                             | What it does                                          | What it doesn’t        |
|------------------------|-------------------------------------|-------------------------------------------------------------|-------------------------------------------------------|------------------------|
| Rejection or filtering | The accepted dataset or rollout set | Before a later training step, or before returning an answer | Removes low-scoring samples                           | Update the policy      |
| Search or selection    | The chosen output at inference time | While generating, ranking, or selecting candidates          | Improves answers w/o finetuning                       | Improve model weights  |
| Policy optimization    | The model distribution              | During training, through rewards and advantages             | Amortizes verifier-preferred behavior into the policy | Prevent reward hacking |

## B Practical verifier design checklist

| Question                                        | Why it matters                          | Failure if ignored                              | Book anchor       |
|-------------------------------------------------|-----------------------------------------|-------------------------------------------------|-------------------|
| What object is verified?                        | Fixes the reward interface.             | The claim exceeds the evidence.                 | Chapter 2         |
| What evidence is consumed?                      | Makes the verifier auditable.           | Hidden inputs shape reward silently.            | Chapter 2         |
| What remains off-screen?                        | Bounds the claim.                       | The model optimizes unmeasured behavior.        | Chapters 7, 10    |
| Is the reward binary, graded, or dense?         | Determines credit assignment.           | Correct checks become weak training signal.     | Chapters 3 and 5  |
| Are extraction and normalization specified?     | Prevents format reward.                 | Formatting becomes the task.                    | Chapters 2 and 4  |
| Are hidden tests or audit checks separate?      | Reduces direct overfitting.             | The policy learns the test suite.               | Chapter 7         |
| Has the high-reward tail been audited?          | Tests optimizer-selected outputs.       | Best-of- $N$ amplifies exploits.                | Chapters 6 and 7  |
| Is verifier drift tracked across checkpoints?   | Measures validity under optimization.   | The verifier is only valid for the base policy. | Chapter 10        |
| Are tools and environment state logged?         | Makes agentic reward debuggable.        | Harness bugs become reward hacks.               | Chapter 9         |
| Is the verifier versioned?                      | Makes results reproducible.             | Patches cannot be tied to behavior changes.     | Chapters 7 and 10 |
| Is test-time search separated from policy gain? | Clarifies what improved.                | A system gain is misreported as model learning. | Chapters 6 and 10 |
| Are tax regressions checked?                    | Prevents narrow benchmark optimization. | Accuracy rises while deployment quality falls.  | Chapter 10        |



## C Core Terminology

### C.1 Reward and verifier terms

- **Verifiable reward:** A reward signal derived from an outcome, execution, proof state, trace, or other artifact that can be checked with reasonable reliability.
- **Verifier:** The mechanism that performs that check, whether symbolic, executable, formal, learned, or hybrid.
- **Reward signal:** The scalar or graded feedback that learning actually sees after verification.
- **Interface:** The part of the task exposed to checking, such as a final answer, a program, a proof state, a citation set, or an environment trajectory.
- **Outcome verifier:** A checker that scores a completed solution rather than its intermediate steps.
- **Process reward:** A reward signal attached to intermediate reasoning, subgoals, or partial traces rather than only the final artifact.
- **Programmatic verifier:** A rule-based checker implemented through deterministic logic, execution, or formal constraints.
- **Learned verifier:** A model-based judge that predicts correctness, quality, or consistency.
- **Verifier stack:** A layered pipeline that combines multiple checks before producing a reward or decision.
- **Signal quality:** How informative, stable, and hard to game the reward is for the capability of interest.

### C.2 Evaluation, reporting, and robustness terms

- **Base model and checkpoint:** The starting policy and exact checkpoint used before RLVR or test-time intervention.
- **RLVR model and checkpoint:** The trained policy and exact checkpoint whose improvement is being claimed.
- **Training verifier:** The verifier whose score is used as reward during RLVR.
- **Evaluation verifier:** The verifier or benchmark grader used to report results after training.
- **Independent audit verifier:** A separate checker, judge, harness, or human audit path used to test whether the policy overfit the training verifier.
- **Audit suite:** A held-out set of checks, examples, adversarial probes, or human review items used to test the verifier itself.
- **High-reward tail:** The region of outputs selected by best-of- $N$ , search, or gradient optimization because the verifier scores them highly.
- **pass@1:** Single-sample policy quality under the specified decoding setup.
- **pass@ $N$ :** Policy plus sampling; the probability that at least one of  $N$  samples passes the checker.
- **Search-guided score:** A system-level score that includes active test-time control, reranking, verifier-guided search, or tool-mediated exploration.
- **Generation budget:** The number of samples, tokens, tool calls, search steps, or other generation-side work used to produce the answer.
- **Verification budget:** The number or cost of tests, judge calls, proof checks, environment runs, or other verifier-side work used to score or select the answer.
- **Hidden-test status:** Whether the policy trained against the same tests being reported, and whether a separate hidden or independent suite was used.

- **Contamination audit:** Checks for prompt, answer, partial-prompt, trace, and test leakage across pretraining, SFT, preference data, RL data, and synthetic data construction.
- **Regression tax:** Calibration, abstention, refusal, safety, instruction-following, latency, or cost regressions paid elsewhere while improving the target score.
- **Faithfulness:** The extent to which an externalized explanation tracks the causal basis of the model’s answer.
- **Calibration:** The relationship between expressed confidence and actual correctness.

### C.3 Agentic terms

- **Harness:** The environment-backed interface that turns a model rollout into trainable and auditable data, including task inputs, tool access, state, execution, logging, and reward computation.
- **Deployment interface:** The real interface the system will use at serving time, including tools, state, latency limits, user inputs, and logging.

Ahmadian, Arash, Chris Cremer, Matthieu Gall , et al. 2024. “Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs.” *arXiv Preprint arXiv:2402.14740*. <https://arxiv.org/abs/2402.14740>.

Allen Institute for AI. 2025. “Olmo 3: Charting a Path Through the Model Flow to Lead Open-Source AI.” November. <https://allenai.org/blog/olmo3>.

Anthropic. 2025. “Writing Effective Tools for Agents – with Agents.” September 11. <https://www.anthropic.com/engineering/writing-tools-for-agents>.

Baker, Bowen, Joost Huizinga, Leo Gao, et al. 2025. “Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation.” *arXiv Preprint arXiv:2503.11926*. <https://arxiv.org/abs/2503.11926>.

Bengio, Yoshua, J r me Louradour, Ronan Collobert, and Jason Weston. 2009. “Curriculum Learning.” *Proceedings of the 26th Annual International Conference on Machine Learning*, 41–48. <https://doi.org/10.1145/1553374.1553380>.

Brown, William. 2025a. *Granular Format Rewards for Eliciting Mathematical Reasoning Capabilities in Small Language Models*. GitHub Gist. <https://gist.github.com/willcbb/4676755236bb08cab5f4e54a0475d6fb>.

Brown, William. 2025b. *Verifiers: Environments for LLM Reinforcement Learning*. Software. <https://github.com/PrimeIntellect-ai/verifiers>.

Chen, Mark, Jerry Tworek, Heewoo Jun, et al. 2021. “Evaluating Large Language Models Trained on Code.” *arXiv Preprint arXiv:2107.03374*. <https://arxiv.org/abs/2107.03374>.

Cobbe, Karl, Vineet Kosaraju, Mohammad Bavarian, et al. 2021. “Training Verifiers to Solve Math Word Problems.” *arXiv Preprint arXiv:2110.14168*. <https://arxiv.org/abs/2110.14168>.

Coste, Thomas, Usman Anwar, Robert Kirk, and David Krueger. 2023. “Reward Model Ensembles

- Help Mitigate Overoptimization.” *arXiv Preprint arXiv:2310.02743*. <https://arxiv.org/abs/2310.02743>.
- Da, Jeff, Clinton Wang, Xiang Deng, Yuntao Ma, Nikhil Barhate, and Sean Hendryx. 2025. “Agent-RLVR: Training Software Engineering Agents via Guidance and Environment Rewards.” *arXiv Preprint arXiv:2506.11425*. <https://arxiv.org/abs/2506.11425>.
- DeepSeek-AI, Daya Guo, Dejian Yang, et al. 2025. “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.” *arXiv Preprint arXiv:2501.12948*. <https://arxiv.org/abs/2501.12948>.
- Gao, Leo, John Schulman, and Jacob Hilton. 2023. “Scaling Laws for Reward Model Overoptimization.” *Proceedings of the 40th International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2210.10760>.
- Jackson, Jacob, Ben Trapani, Nathan Wang, and Wanqi Zhu. 2026. *Improving Composer Through Real-Time RL*. Cursor Blog. <https://cursor.com/blog/real-time-rl-for-composer>.
- Jain, Naman, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. 2025. “R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents.” *arXiv Preprint arXiv:2504.07164*. <https://arxiv.org/abs/2504.07164>.
- Khan, Zohaib, Omer Tafveez, and Zoha Hayat Bhatti. 2026. “Plasticity Vs. Rigidity: The Impact of Low-Rank Adapters on Reasoning on a Micro-Budget.” *arXiv Preprint arXiv:2601.06677*. <https://arxiv.org/abs/2601.06677>.
- Kojima, Takeshi, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. “Large Language Models Are Zero-Shot Reasoners.” *arXiv Preprint arXiv:2205.11916*. <https://arxiv.org/abs/2205.11916>.
- Kydlicek, Hynek. 2025. *Math-Verify: Math Verification Library*. V. 0.6.1. Released. <https://github.com/huggingface/Math-Verify>.
- Lambert, Nathan, Jacob Morrison, Valentina Pyatkin, et al. 2024. “Tulu 3: Pushing Frontiers in Open Language Model Post-Training.” *arXiv Preprint arXiv:2411.15124*. <https://arxiv.org/abs/2411.15124>.
- Lambert, Nathan, Valentina Pyatkin, Jacob Morrison, et al. 2024. “RewardBench: Evaluating Reward Models for Language Modeling.” *arXiv Preprint arXiv:2403.13787*. <https://arxiv.org/abs/2403.13787>.
- Lanham, Tamera, Anna Chen, Ansh Radhakrishnan, et al. 2023. “Measuring Faithfulness in Chain-of-Thought Reasoning.” *arXiv Preprint arXiv:2307.13702*. <https://arxiv.org/abs/2307.13702>.
- Le, Hung, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. 2022. “CodeRL: Mastering Code Generation Through Pretrained Models and Deep Reinforcement Learning.” *arXiv Preprint arXiv:2207.01780*. <https://arxiv.org/abs/2207.01780>.
- Lightman, Hunter, Vineet Kosaraju, Yura Burda, et al. 2023. “Let’s Verify Step by Step.” *arXiv*

- Preprint *arXiv:2305.20050*. <https://arxiv.org/abs/2305.20050>.
- Liu, Haochen, Bo Yuan, Yao Hu, et al. 2023. “Reinforcement Learning from Unit Test Feedback.” *arXiv Preprint arXiv:2307.04349*. <https://arxiv.org/abs/2307.04349>.
- Liu, Jiawei, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. “Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation.” *arXiv Preprint arXiv:2305.01210*. <https://arxiv.org/abs/2305.01210>.
- Lu, Pan, Hritik Bansal, Tony Xia, et al. 2023. “MathVista: Evaluating Mathematical Reasoning of Foundation Models in Visual Contexts.” *arXiv Preprint arXiv:2310.02255*. <https://arxiv.org/abs/2310.02255>.
- Nye, Maxwell, Anders Johan Andreassen, Guy Gur-Ari, et al. 2021. “Show Your Work: Scratchpads for Intermediate Computation with Language Models.” *arXiv Preprint arXiv:2112.00114*. <https://arxiv.org/abs/2112.00114>.
- OpenAI. 2024. “Learning to Reason with LLMs.” September 12. <https://openai.com/index/learning-to-reason-with-llms/>.
- OpenAI. 2026. “Graders.” <https://developers.openai.com/api/docs/guides/graders>.
- Pan, Alexander, Kush Bhatia, and Jacob Steinhardt. 2022. “The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models.” *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2201.03544>.
- Patel, Dwarkesh. 2025a. “Andrej Karpathy — AGI Is Still a Decade Away.” October 17. <https://www.dwarkesh.com/p/andrej-karpathy>.
- Patel, Dwarkesh. 2025b. *RL Is Even More Information Inefficient Than You Thought*. Dwarkesh Podcast. <https://www.dwarkesh.com/p/bits-per-sample>.
- Peng, Hao, Yunjia Qi, Xiaozhi Wang, Bin Xu, Lei Hou, and Juanzi Li. 2025. “VerIF: Verification Engineering for Reinforcement Learning in Instruction Following.” *arXiv Preprint arXiv:2506.09942*. <https://arxiv.org/abs/2506.09942>.
- Piché, Alexandre, Ehsan Kamalloo, Rafael Pardini, Xiaoyin Chen, and Dzmitry Bahdanau. 2025. “PipelineRL: Faster on-Policy Reinforcement Learning for Long Sequence Generation.” *arXiv Preprint arXiv:2509.19128*. <https://arxiv.org/abs/2509.19128>.
- Quiroz-Gutierrez, Marco. 2025. “Meta Is Reportedly Scrambling Multiple ‘War Rooms’ of Engineers to Figure Out How DeepSeek’s AI Is Beating Everyone Else at a Fraction of the Price.” January 27. <https://fortune.com/2025/01/27/meta-war-rooms-engineers-deepseek-ai/>.
- rLLM Project. 2026. “DeepSWE Software Engineering Agent.” <https://docs.rllm-project.com/examples/deep-swe>.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. “Proximal Policy Optimization Algorithms.” *arXiv Preprint arXiv:1707.06347*. <https://arxiv.org/abs/1707.06347>.

- Shao, Zhihong, Peiyi Wang, Qihao Zhu, et al. 2024. “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.” *arXiv Preprint arXiv:2402.03300*. <https://arxiv.org/abs/2402.03300>.
- Sheng, Guangming, Chi Zhang, Zilingfeng Ye, et al. 2024. “HybridFlow: A Flexible and Efficient RLHF Framework.” *arXiv Preprint arXiv:2409.19256*. <https://arxiv.org/abs/2409.19256>.
- Shojaee, Parshin, Mehrdad Li, Arman Cohan, Hannaneh Hajishirzi, Wen-tau Yih, and Sameer Singh. 2023. “PPOCoder: The Effectiveness of Reinforcement Learning for Code Generation.” *arXiv Preprint arXiv:2301.13816*. <https://arxiv.org/abs/2301.13816>.
- Skalse, Joar, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. 2022. “Defining and Characterizing Reward Hacking.” *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2209.13085>.
- Snell, Charlie, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. “Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters.” *arXiv Preprint arXiv:2408.03314*. <https://arxiv.org/abs/2408.03314>.
- Sullivan, Aidan, and Daphne Koller. 2025. “GRPO Is Secretly a Process Reward Model.” *arXiv Preprint arXiv:2509.21154*. <https://arxiv.org/abs/2509.21154>.
- Sun, Lin, Chuang Liu, Xiaofeng Ma, Tao Yang, Weijia Lu, and Ning Wu. 2025. “FreePRM: Training Process Reward Models Without Ground Truth Process Labels.” *arXiv Preprint arXiv:2506.03570*, ahead of print. <https://doi.org/10.48550/arXiv.2506.03570>.
- Tan, Sijun, Michael Luo, Colin Cai, et al. 2025. *rLLM: A Framework for Post-Training Language Agents*. Notion Blog. <https://github.com/rllm-org/rllm>.
- Team OLMo, Allyson Ettinger, Amanda Bertsch, et al. 2025. “Olmo 3.” *arXiv Preprint arXiv:2512.13961*. <https://arxiv.org/abs/2512.13961>.
- TMTPOST Global. 2025. “DeepSeek Panic: How Silicon Valley Giants Are Responding to China’s AI Disruptor.” January 30. <https://www.tmtpost.com/7445592.html>.
- Trinh, Trieu H., Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. 2025. “Solving Olympiad Geometry Without Human Demonstrations.” *arXiv Preprint arXiv:2510.01346*. <https://arxiv.org/abs/2510.01346>.
- Turpin, Miles, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. “Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting.” *arXiv Preprint arXiv:2305.04388*. <https://arxiv.org/abs/2305.04388>.
- Uesato, Jonathan, Nate Kushman, Ramana Kumar, et al. 2022. “Solving Math Word Problems with Process- and Outcome-Based Feedback.” *arXiv Preprint arXiv:2211.14275*. <https://arxiv.org/abs/2211.14275>.

- Wang, Peiyi, Lei Li, Zhihong Shao, et al. 2024. “Math-Shepherd: Verify and Reinforce LLMs Step-by-Step Without Human Annotations.” *arXiv Preprint arXiv:2312.08935*. <https://arxiv.org/abs/2312.08935>.
- Wang, Xuezhi, Jason Wei, Dale Schuurmans, et al. 2022. “Self-Consistency Improves Chain of Thought Reasoning in Language Models.” *arXiv Preprint arXiv:2203.11171*. <https://arxiv.org/abs/2203.11171>.
- Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. 2022. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” *arXiv Preprint arXiv:2201.11903*. <https://arxiv.org/abs/2201.11903>.
- Werra, Leandro von, Younes Belkada, Lewis Tunstall, et al. 2020. *TRL: Transformers Reinforcement Learning*. Released. <https://github.com/huggingface/trl>.
- Xie, Tianbao, Danyang Zhang, Jixuan Chen, et al. 2024. “OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments.” *arXiv Preprint arXiv:2404.07972*. <https://arxiv.org/abs/2404.07972>.
- Xin, Huajian, Daya Guo, Zhihong Shao, et al. 2024. “DeepSeek-Prover: Advancing Theorem Proving in LLMs Through Large-Scale Synthetic Data.” *arXiv Preprint arXiv:2405.14333*. <https://arxiv.org/abs/2405.14333>.
- Xin, Huajian, Z. Z. Ren, Junxiao Song, et al. 2024. “DeepSeek-Prover-V1.5: Harnessing Proof Assistant Feedback for Reinforcement Learning and Monte-Carlo Tree Search.” *arXiv Preprint arXiv:2408.08152*. <https://arxiv.org/abs/2408.08152>.
- Yuan, Lifan et al. 2024. “Free Process Rewards Without Process Labels.” *arXiv Preprint arXiv:2412.01981*. <https://arxiv.org/abs/2412.01981>.
- Zhang, Jiajie, Yushi Bai, Xin Lv, et al. 2024. “LongCite: Enabling LLMs to Generate Fine-Grained Citations in Long-Context QA.” *arXiv Preprint arXiv:2409.02897*. <https://arxiv.org/abs/2409.02897>.
- Zhang, Lunjun, Arian Hosseini, Hritik Bansal, Seyed Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. 2025. “Generative Verifiers: Reward Modeling as Next-Token Prediction.” *Proceedings of the International Conference on Learning Representations (ICLR)*, ahead of print. <https://doi.org/10.48550/arXiv.2408.15240>.
- Zhang, Sheng, Qianchu Liu, Guanghui Qin, Tristan Naumann, and Hoifung Poon. 2025. “Med-RLVR: Emerging Medical Reasoning from a 3B Base Model via Reinforcement Learning.” *arXiv Preprint arXiv:2502.19655*. <https://arxiv.org/abs/2502.19655>.
- Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, et al. 2023. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.” *arXiv Preprint arXiv:2306.05685*. <https://arxiv.org/abs/2306.05685>.
- Zhou, Shuyan, Frank F. Xu, Hao Zhu, et al. 2023. “WebArena: A Realistic Web Environment for Building Autonomous Agents.” *arXiv Preprint arXiv:2307.13854*. <https://arxiv.org/abs/2307.13854>.